



ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ

Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών υπολογιστών

Ροή Σ: Σήματα, Έλεγχος και Ρομποτική

ΑΝΑΓΝΩΡΙΣΗ ΠΡΟΤΥΠΩΝ

2η Εργαστηριακή Άσκηση

«Αναγνώριση Είδους και Εξαγωγή Συναισθήματος από Μουσική»

Στοιχεία Ομάδας

- Μέλος 1^ο: Πέππας Μιχαήλ-Αθανάσιος | Α.Μ: 03121026
- Μέλος 2^ο: Αυγερινός Παναγιώτης | Α.Μ: 03121023
- Ημερομηνία Παράδοσης Αναφοράς: 08.12.2025

■ Εισαγωγή

Έχοντας λάβει υπόψιν το πλαίσιο και τις απαιτήσεις της 2ης Εργαστηριακής Άσκησης, όπως αυτές καθορίζονται στο συνοδευτικό έγγραφο (PDF) της εκφώνησης, υποβάλλουμε τις λύσεις μας σε δύο διακριτά αρχεία:

1. **patrec2_03121026_03121023_report.pdf:** Η παρούσα αναφορά, η οποία περιέχει την αναλυτική παρουσίαση των λύσεων, τις απαντήσεις στα θεωρητικά ερωτήματα και τον σχολιασμό των αποτελεσμάτων για όλα τα ζητούμενα (Βήματα 0-10).
2. **patrec2_03121026_03121023_code.py:** Το αρχείο κώδικα (σε μορφή .py, προερχόμενο από το .ipynb notebook), το οποίο περιέχει την πλήρη υλοποίηση των λύσεων των ερωτημάτων. Ο κώδικας είναι πλήρως σχολιασμένος (με comments και Markdown cells) στα κρίσιμα σημεία.

Στη συνέχεια, παρατίθενται οι απαντήσεις μας, οργανωμένες ανά βήμα και ερώτημα, σύμφωνα με τη σειρά που τέθηκαν.

▪ **Βήμα 0: Εξοικείωση με Kaggle kernels**

Αρχικά επιβεβαιώθηκαν σε ένα pre-step του βήματος πως τα notebooks που δίνονται υπάρχουν και μπορούν να φορτωθούν. Έπειτα, ελέγξαμε πως το dataset της εργαστηριακής άσκησης έχει συνδεθεί σωστά στο περιβάλλον του Kaggle και ότι μπορούμε να δούμε τη βασική του δομή. Σύμφωνα με την εκφώνηση, χρησιμοποιήθηκαν οι εντολές:

```
os.listdir("../input/patreco3-multitask-affective-music/data/")  
  
data_root = "../input/patreco3-multitask-affective-music/data/"  
  
και print(os.listdir(data_root))
```

ώστε να εμφανιστεί η λίστα με όλα τα στοιχεία (φακέλους και αρχεία) που περιέχονται στον συγκεκριμένο φάκελο.

▪ **Βήμα 1: Εξοικείωση με φασματογραφήματα στην κλίμακα mel**

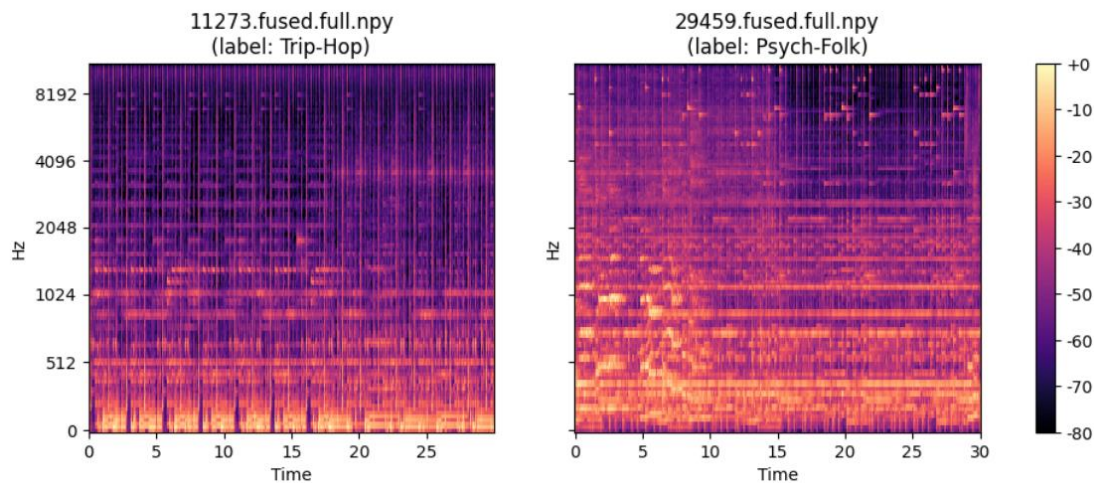
Πρώτα, υλοποιήθηκε μία συνάρτηση που εντοπίζει κοινά αρχεία στα fma_genre_spectrogram/fma_genre_spectrogram_beat, διότι θα χρειαστεί στη συνέχεια ώστε να μπορούμε αργότερα να συγκρίνουμε ακριβώς τα ίδια κομμάτια.

α) Διαβάζοντας το fma_genre_spectrograms/train_labels.txt, δημιουργήσαμε αντιστοιχίσεις από το track_id τόσο προς το όνομα του αρχείου φασματογραφήματος (από τον φάκελο fma_genre_spectrograms/train) όσο και προς το label.

Επιλέξαμε το track με ID = 11273 και label Trip-Hop και το track με ID = 29459 και label Psych-Folk. Τα αντίστοιχα αρχεία φασματογραμμάτων είναι 11273.fused.full.npy και 29459.fused.full.npy.

β) Ακολουθώντας τον κώδικα του βοηθητικού notebook [0], φορτώσαμε το αντίστοιχο αρχείο φασματογραφήματος από τον φάκελο fma_genre_spectrograms/train., θεωρήσαμε ότι οι πρώτες 128 γραμμές αντιστοιχούν στα mel bins και οι επόμενες γραμμές στο chroma. Σε κάθε περίπτωση, είναι ο αριθμός των ζωνών συχνοτήτων στην κλίμακα Mel είναι 128 και 1291/1293 ο αριθμός των χρονικών frames. Άρα, κάθε κομμάτι αναπαρίσταται ως μία ακολουθία περίπου 1290 διανυσμάτων 128 διαστάσεων.

γ) Με χρήση της συνάρτησης librosa.display.specshow τα δύο mel φασματογραφήματα φαίνονται παρακάτω:



Η πληροφορία που μας δίνουν τα διαγράμματα είναι πρακτικά πόση ενέργεια έχουν τα σήματα την εκάστοτε χρονική στιγμή και συχνότητα.

Παρατηρώντας τα αποτελέσματα, το Trip-Hop παρουσιάζει έντονες κάθετες δομές και μεγάλη ενέργεια στις χαμηλές και μεσαίες συχνότητες, κάτι που σχετίζεται με ρυθμικά beats, μπάσο και percussive στοιχεία του. Αντίθετα, το κομμάτι Psych-Folk εμφανίζει πιο ομοιόμορφη κατανομή ενέργειας κυρίως στις μεσαίες συχνότητες και πιο ομαλές οριζόντιες ζώνες, που αντιστοιχούν πιθανότατα σε συγχορδίες και μελωδικές γραμμές όπως φωνητικά.

δ) Η κλίμακα Mel είναι μια ψυχοακουστική κλίμακα συχνοτήτων που σχεδιάστηκε ώστε να προσεγγίζει τον τρόπο με τον οποίο το ανθρώπινο αυτί αντιλαμβάνεται τις διαφορές τόνου. Σε χαμηλές συχνότητες, μικρές διαφορές γίνονται εύκολα αντιληπτές, ενώ σε πολύ υψηλές συχνότητες απαιτείται μεγαλύτερη μεταβολή για να γίνει αισθητή κάποια αλλαγή.

Η κλίμακα Mel υλοποιεί αυτή τη μη γραμμική συμπεριφορά με μια συνάρτηση μετασχηματισμού, τυπικά της μορφής $\text{mel}(f) = 2595 * \log_{10}(1 + f / 700)$. Η σχέση βρέθηκε εμπειρικά από μετρήσεις σε ανθρώπους.

Στην επεξεργασία μουσικών σημάτων χρησιμοποιούμε συνήθως Mel-spectrograms και αντλούνται οι MFCCs (Mel-Frequency Cepstral Coefficients) συντελεστές από τα σήματα, ένα σύνολο χαρακτηριστικών που περιγράφουν το φάσμα του. Έτσι, οι συχνότητες κωδικοποιούνται με τρόπο πιο κοντά στην ανθρώπινη αντίληψη και τα εξαγόμενα χαρακτηριστικά είναι πιο κατάλληλα αναγνώριση προτύπων.

Βήμα 2: Συγχρονισμός φασματογραφημάτων στο ρυθμό της μουσικής (beat-synced spectrograms)

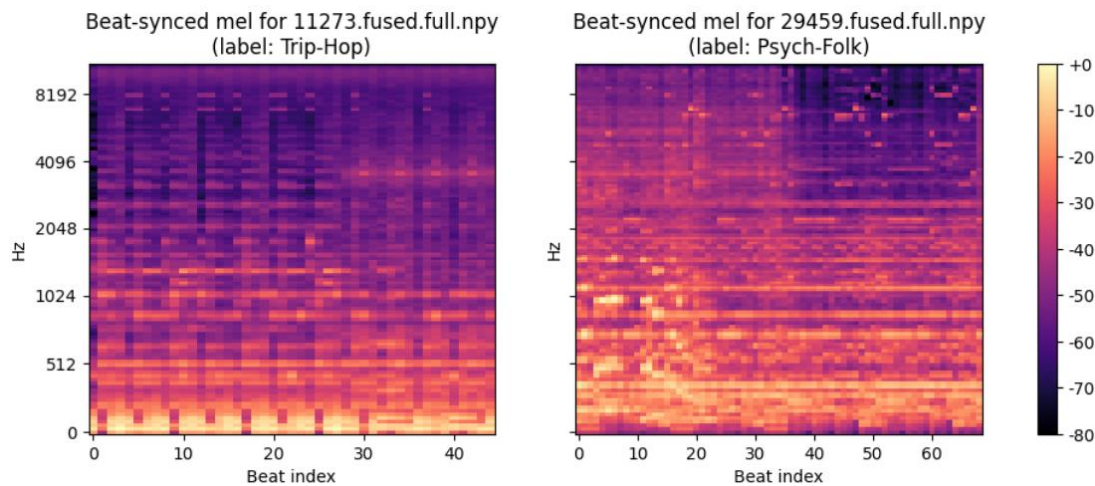
α) Στο υποερώτημα 2α χρησιμοποιούμε τα ίδια φασματογραφήματα Mel που υπολογίσαμε στο Βήμα 1 για τα δύο επιλεγμένα κομμάτια. Για το κομμάτι Trip-Hop με `track_id = 11273` το φασματογράφημα έχει διαστάσεις (128, 1291), ενώ για το κομμάτι Psych-Folk με `track_id = 29459` έχει διαστάσεις (128, 1293).

Και στις δύο περιπτώσεις, οι 128 γραμμές αντιστοιχούν σε ζώνες συχνοτήτων στην κλίμακα Mel, ενώ οι 1291/1293 στήλες αντιστοιχούν σε διαδοχικά χρονικά frames. Αυτό σημαίνει ότι κάθε κομμάτι περιγράφεται από περίπου 1300 χρονικά βήματα.

Για ένα LSTM αυτό είναι σχετικά μεγάλο μήκος ακολουθίας: αυξάνει σημαντικά τον χρόνο εκπαίδευσης και την απαιτούμενη μνήμη, ενώ κάνει δυσκολότερη τη διάδοση των βαθμίδων (vanishing/exploding gradients), ειδικά αν χρησιμοποιηθούν περισσότερα επίπεδα ή αν το μοντέλο πρέπει να «θυμάται» πληροφορία σε μεγάλη χρονική κλίμακα.

Άρα, με τα αρχικά φασματογραφήματα δεν είναι ιδιαίτερα αποδοτικό να εκπαιδεύσουμε ένα LSTM, καθώς κάθε παράδειγμα είναι ακολουθία 1291–1293 βημάτων.

β) Χρησιμοποιώντας τα αρχεία που δίνονται, για κάθε ένα από τα δύο κομμάτια εντοπίζουμε το αντίστοιχο beat_synced αρχείο και υπολογίζουμε την ενέργεια ανά χρονικό frame. Αφαιρούμε τα τελικά frames που είναι ουσιαστικά μηδενικά (padding), ώστε να μείνει μόνο η έγκυρη ακολουθία των beat frames. Τα φασματογραφήματα αυτά, φαίνονται ακολούθως:



Για το κομμάτι Trip-Hop (11273) το beat-synced φασματογράφημα Mel έχει τελική μειωμένη διάσταση (128, 45), ενώ για το Psych-Folk (29459) η διάσταση είναι (128, 69). Δηλαδή, ο αριθμός των χρονικών βημάτων μειώνεται δραστικά: από περίπου 1290 short-time frames σε μόλις 45 και 69 beat frames αντίστοιχα (περίπου 20–30 φορές μικρότερες ακολουθίες).

Οπτικά, τα beat-synced φασματογραφήματα διατηρούν τη γενική κατανομή ενέργειας σε συχνότητα: στο Trip-Hop κομμάτι παραμένουν οι έντονες χαμηλές/μεσαίες συχνότητες και τα ρυθμικά μοτίβα, ενώ στο Psych-Folk κομμάτι εξακολουθούμε να βλέπουμε τις ισχυρές μεσαίες συχνότητες και τις πιο «οριζόντιες» δομές που αντιστοιχούν σε συγχορδίες.

Ωστόσο, ο οριζόντιος άξονας είναι πλέον beats, άρα κάθε στήλη συμπυκνώνει την πληροφορία γύρω από ένα ρυθμικό σημείο. Αυτό κάνει τα δεδομένα πολύ πιο κατάλληλα για ένα LSTM.

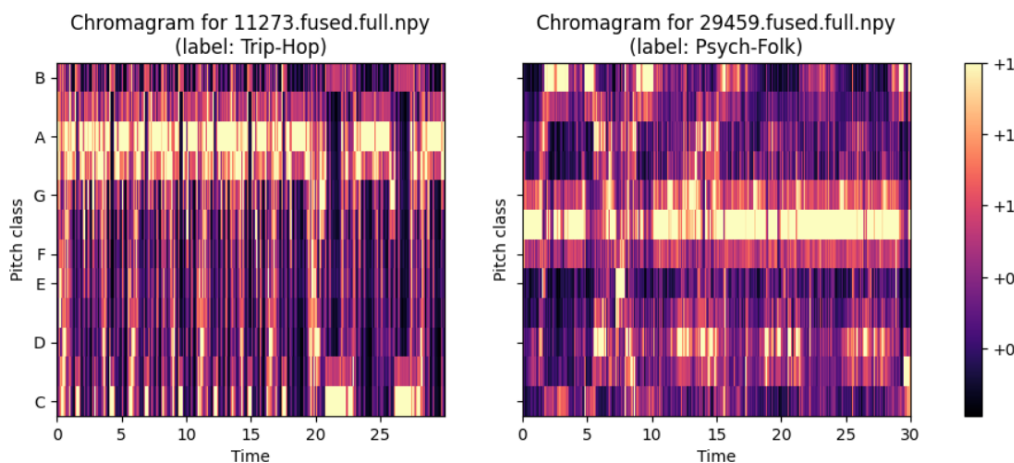
▪ **Βήμα 3:** Εξουκείωση με χρωμογραφήματα

Επαναλαμβάνουμε την ανάλυση των Βημάτων 1 και 2, αλλά αυτή τη φορά για τα χρωμογραφήματα, τα οποία αποτυπώνουν την ενέργεια του σήματος σε 12 ζώνες που αντιστοιχούν στις νότες της δυτικής μουσικής (C, C#, D, ..., B).

α) Σύμφωνα με τον βοηθητικό κώδικα εξάγουμε το χρωμογράφημα κρατώντας τις γραμμές από το index 128 και πάνω για τα κομμάτια 11273.fused.full.npy – Trip-Hop και 29459.fused.full.npy – Psych-Folk

Για το κομμάτι Trip-Hop προκύπτει χρωμογράφημα διαστάσεων (12, 1291), ενώ για το Psych-Folk η διάσταση είναι (12, 1293). Οι 12 γραμμές αντιστοιχούν στις 12 κλάσεις ύψους και οι ~1290 στήλες σε διαδοχικά χρονικά frames. Άρα, όπως και στα φασματογραφήματα Mel, κάθε τραγούδι αναπαρίσταται ως μια ακολουθία ~1300 χρονικών βημάτων, αλλά τώρα η έμφαση είναι στην κατανομή ενέργειας ανά νότα.

β) Τα φασματογραφήματα φαίνονται παρακάτω:

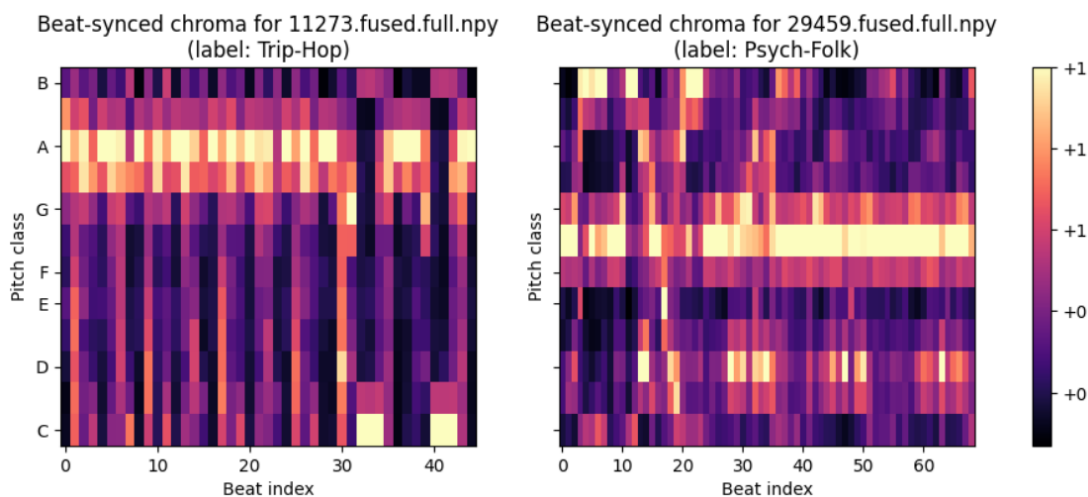


Στο κομμάτι Trip-Hop παρατηρούμε ότι η ενέργεια συγκεντρώνεται σε λίγες συγκεκριμένες pitch classes (π.χ. γύρω από A και G), με έντονες επαναλήψεις στον χρόνο, κάτι που αντανακλά την επανάληψη των συγχορδιών αυτών. Αντίθετα, στο Psych-Folk το χρωμογράφημα εμφανίζει πιο διάσπαρτη ενέργεια σε περισσότερες νότες και πιο ομαλές οριζόντιες ζώνες, κάτι που υποδηλώνει πολυπλοκότερες αλλαγές σε συγχορδίες-νότες.

γ) Στη συνέχεια, επαναλαμβάνουμε το Βήμα 2 για τα χρωμογραφήματα, χρησιμοποιώντας τα beat-synced fused φασματογραφήματα από τον φάκελο fma_genre_spectrograms_beat/train. Εργαζόμαστε όπως πριν.

Το κομμάτι Trip-Hop το beat-synced χρωμογράφημα έχει τελική διάσταση (12, 45), ενώ για το Psych-Folk η διάσταση είναι (12, 69).

δ) Το φασματογράφημα φαίνεται παρακάτω:



Βλέπουμε ότι η βασική αρμονική δομή των δύο τραγουδιών διατηρείται: στο Trip-Hop κομμάτι παραμένει η έντονη ενεργοποίηση λίγων pitch classes σε πολλά διαδοχικά beats, ενώ στο Psych-Folk παρατηρούμε πιο σύνθετα μοτίβα με ενεργοποίηση διαφορετικών pitches σε μεγαλύτερα χρονικά διαστήματα.

Ομοίως, η βασική διαφορά είναι ότι πλέον κάθε στήλη αντιστοιχεί σε ένα beat. Αυτό σημαίνει ότι οι ακολουθίες είναι πιο ενωμένες, και όπως προαναφέρθηκε αυτό είναι γεγονός που καθιστά τα beat_synced χρωμογραφήματα ιδιαίτερα κατάλληλα ως είσοδο σε LSTM.

▪ Βήμα 4: Φόρτωση και ανάλυση δεδομένων

α) Στο πρώτο υποερώτημα μελετάμε την κλάση `SpectrogramDataset` που μας δίνεται στον βοηθητικό κώδικα. Δημιουργούμε ένα αντικείμενο `train_dataset` πάνω στα αρχικά φασματογραφήματα (`fma_genre_spectrograms`) με `class_mapping=CLASS_MAPPING` και `train=True` και εξετάζουμε το στοιχείο `train_dataset[10]`. Η κλάση υλοποιεί ένα PyTorch Dataset που:

- διαβάζει από τον δίσκο τα `.npy` αρχεία φασματογραμμάτων,
- εφαρμόζει την κατάλληλη επιλογή χαρακτηριστικών (`mel`, `chroma` ή `fused`, ανάλογα με το `feat_type`),
- κρατάει την διάρκεια της ακολουθίας και, αν χρειαστεί, πραγματοποιεί `padding`
- επιστρέφει ένα στοιχείο της μορφής (`spectrogram`, `label`, `original_length`).

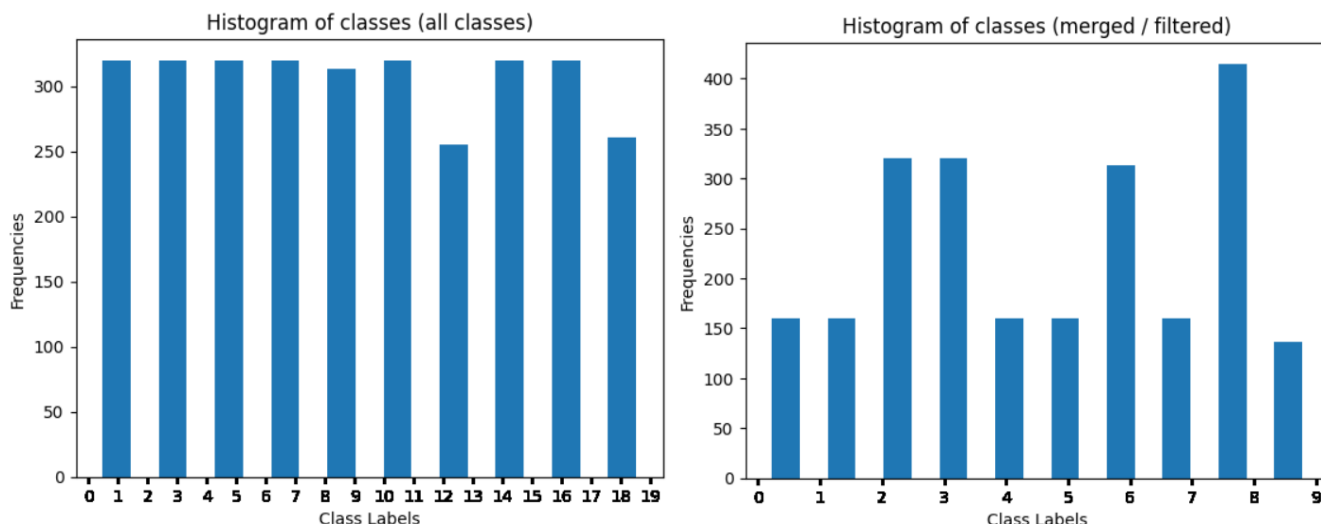
Στη δική μας εκτύπωση βλέπουμε ότι το στοιχείο 10 έχει τη μορφή (`..., 0, 1291`): το φασματογράφημα είναι ένας πίνακας πραγματικών τιμών (σε dB) με σχήμα (1293, 128), η ακέραια ετικέτα είναι 0 και το `original_length` είναι 1291. Αρα η κλάση αποθηκεύει μεγαλύτερο πίνακα (1293 χρονικά βήματα), αλλά κρατάει ξεχωριστά το πραγματικό μήκος 1291, ώστε να γνωρίζουμε ποιο μέρος αντιστοιχεί σε `valid` δεδομένα και ποιο σε `padding`.

β) Στο υποερώτημα 4β χρησιμοποιούμε δύο εκδοχές του `SpectrogramDataset` για να μελετήσουμε τη διαδικασία συγχώνευσης κλάσεων:

- `train_dataset_all` με `class_mapping=None` (όλες οι αρχικές κλάσεις όπως προκύπτουν από το αρχείο `labels`).
- `train_dataset` με `class_mapping=CLASS_MAPPING`, όπου το λεξικό `CLASS_MAPPING` συγχωνεύει συγγενείς μουσικές κατηγορίες σε λίγες υπερκλάσεις και αφαιρεί κλάσεις με πολύ λίγα δείγματα.

γ) Με τη βοηθητική συνάρτηση `plot_class_histogram` σχεδιάζουμε δύο ιστογράμματα: το πρώτο δείχνει την κατανομή των 20 αρχικών κλάσεων, με σχετικά παρόμοιο αριθμό δειγμάτων ανά κλάση. Το δεύτερο ιστόγραμμα, αφού εφαρμόσουμε το `CLASS_MAPPING`, εμφανίζει 10 νέες κλάσεις (`labels 0–9`).

Παρατηρούμε ότι ο αριθμός των δειγμάτων ανά κλάση είναι τώρα μεγαλύτερος και πιο ισορροπημένος για τις περισσότερες κλάσεις – ορισμένες μάλιστα ξεπερνούν τα 400 δείγματα, επειδή προέρχονται από συγχώνευση πολλών παρόμοιων `genres`. Με αυτόν τον τρόπο αποφεύγουμε κλάσεις με πολύ λίγα δείγματα και απλοποιούμε την ταξινόμηση.



Τέλος, για να προετοιμάσουμε τα δεδομένα για τα νευρωνικά μοντέλα, δημιουργούμε ένα νέο SpectrogramDataset με `feat_type='mel'` και `max_length=150`, εφαρμόζοντας και πάλι το CLASS_MAPPING. Εξασφαλίζουμε ότι όλες οι εισοδοι στο LSTM θα έχουν κοινό μέγεθος, κάτι που διευκολύνει την υλοποίηση και τη βελτιστοποίηση, χωρίς να χάνεται η πληροφορία για το πραγματικό μήκος κάθε ακολουθίας.

▪ Βήμα 5: Αναγνώριση μουσικού είδους με LSTM

Στο Βήμα 5 υλοποιήσαμε και εκπαιδεύσαμε ένα LSTM δίκτυο για την αναγνώριση μουσικού είδους, χρησιμοποιώντας τον κώδικα της προηγούμενης άσκησης (αρχείο lstm.py) και τα PyTorch Datasets που ορίστηκαν στο Βήμα 4. Στον κωδικά μας το ερώτημα 5 χωρίστηκε για λόγους ευκολίας υλοποίησης σε 5.1, 5.2, 5.3 .

α) (5.1)

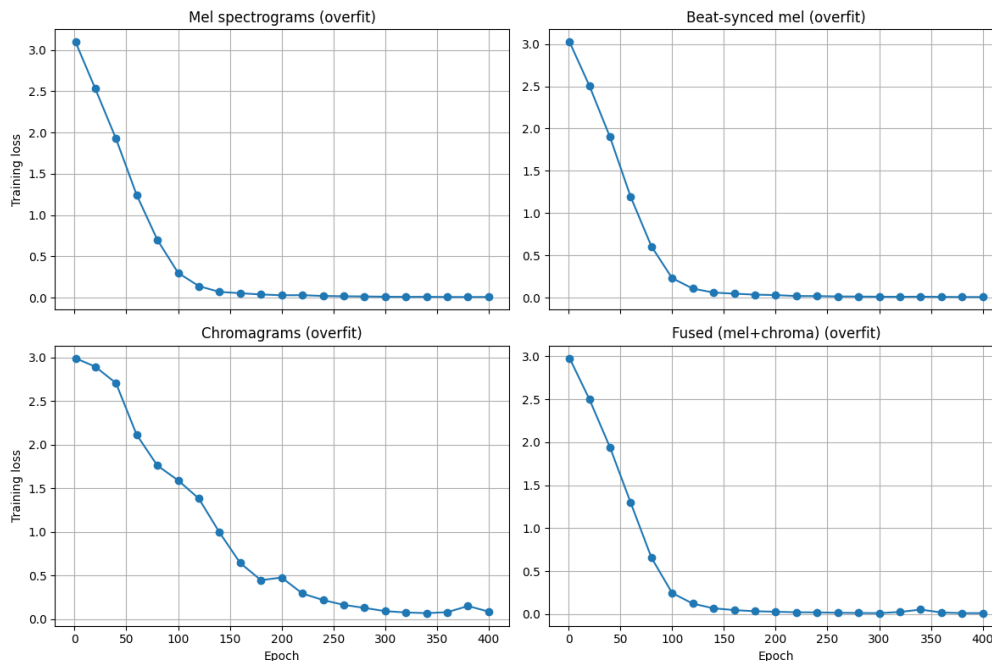
Χρησιμοποιήθηκε η κλάση LSTMBackbone από το αρχείο lstm.py, την οποία ενσωματώνουμε σε μια κλάση Classifier που προσθέτει ένα γραμμικό επίπεδο για παραγωγή logits στις 10 κλάσεις. Ως συνάρτηση κόστους χρησιμοποιούμε nn.CrossEntropyLoss, κατάλληλη για πολυταξική ταξινόμηση.

Τα δεδομένα εισόδου προέρχονται από το SpectrogramDataset ως ακολουθίες διαστάσεων (T, D) , με $T = 150$ χρονικά βήματα (padding/truncation) και D ανάλογα με τα χαρακτηριστικά (128 για mel, 12 για chroma, 140 για fused), μαζί με τα αντίστοιχα μήκη lengths ώστε το LSTM να αξιοποιεί packed sequences.

Τέλος, η βοηθητική συνάρτηση make_loaders κατασκευάζει τα κατάλληλα SpectrogramDataset για κάθε feat_type ('mel', 'chroma', 'fused') και επιστρέφει train/validation DataLoaders, με δυνατότητα χρήσης Subset για ταχύτερο debugging και tuning.

β) 5.2 – Παράμετρος overfit_batch και υπερεκπαίδευση σε λίγα batches

Σύμφωνα με την εκφώνηση, προσθέσαμε στη συνάρτηση train() μια boolean παράμετρο overfit_batch όπως ζητείται. Σκοπός είναι να δούμε αν το LSTM μπορεί να υπερεκπαιδευτεί σε αυτά τα ελάχιστα δείγματα, δηλαδή αν το σφάλμα εκπαίδευσης μπορεί να πλησιάσει το 0.



Τα αποτελέσματα των overfit runs επιβεβαιώνουν τα παραπάνω (διαγράμματα εκτός κώδικα δεν ζητούνται επιπλέον μόνο για την αναφορά). Σημειώνουμε ότι τα νούμερα αυτά είναι cross-entropy losses και όχι ποσοστά. Για 10 κλάσεις, η τυχαία πρόβλεψη αντιστοιχεί περίπου σε $\text{loss} \approx -\log(1/10) \approx 2.30$, άρα η σημαντική πτώση από ~3.0 σε ~0.01 δείχνει ότι το δίκτυο έχει μάθει να προβλέπει σχεδόν σίγουρα τις σωστές κλάσεις στο μικρό αυτό υποσύνολο.

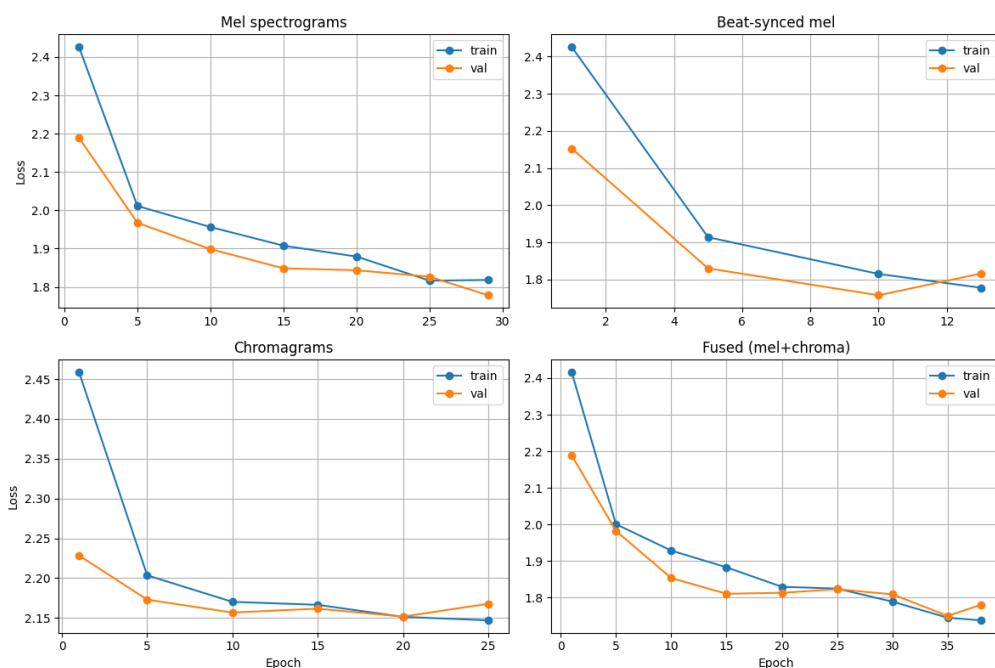
γ-ζ) 5.3 – Εκπαίδευση LSTM για διαφορετικούς τύπους χαρακτηριστικών

Στη συνέχεια, με `overfit_batch=False`, τρέξαμε την κανονική εκπαίδευση για τέσσερις διαφορετικές ρυθμίσεις εισόδων, χρησιμοποιώντας κάθε φορά τα αντίστοιχα datasets του Βήματος 4:

- Φασματογραφήματα Μελ (αρχικό train set)
- Beat-Synced Mel φασματογραφήματα
- Χρωμογραφήματα (chroma)
- Fused φασματογραφήματα (concatenated mel+chroma)

Η συνάρτηση `run_experiment` καλεί `make_loaders` για κάθε `feat_type`, δημιουργεί ένα `LSTMBackbone` με είσοδο `input_dim = D`, κρυφή διάσταση `rnn_size = 128`, `num_layers = 2` και `bidirectional LSTM`, και το εφαρμόζει μέσα σε `Classifier (NUM_CATEGORIES, backbone)`. Το δίκτυο εκπαιδεύεται με Adam ($\text{lr} = 1e-4$) έως 40 εποχές, χρησιμοποιώντας validation set και early stopping (μέσω της κλάσης `EarlyStopper` με `patience=5`).

Σε κάθε εποχή υπολογίζουμε τον μέσο loss εκπαίδευσης και επικύρωσης και τους εκτυπώνουμε περιοδικά. Και εδώ οι τιμές είναι cross-entropy losses, οπότε τιμές κοντά στο 2.3 αντιστοιχούν περίπου σε τυχαίες προβλέψεις, ενώ χαμηλότερες τιμές δείχνουν καλύτερη προσαρμογή. Παρακάτω φαίνονται τα αποτελέσματα για κάποιες εποχές (διαγράμματα μόνο για την αναφορά).

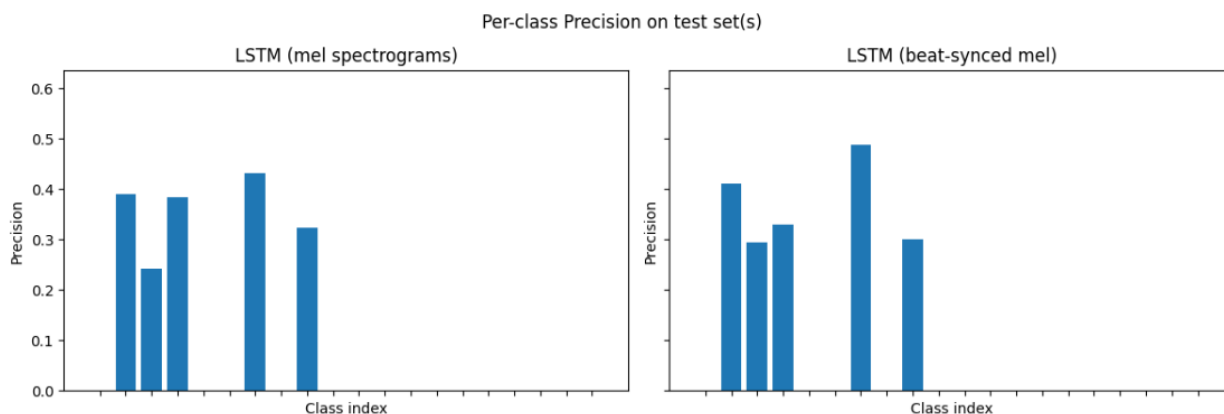


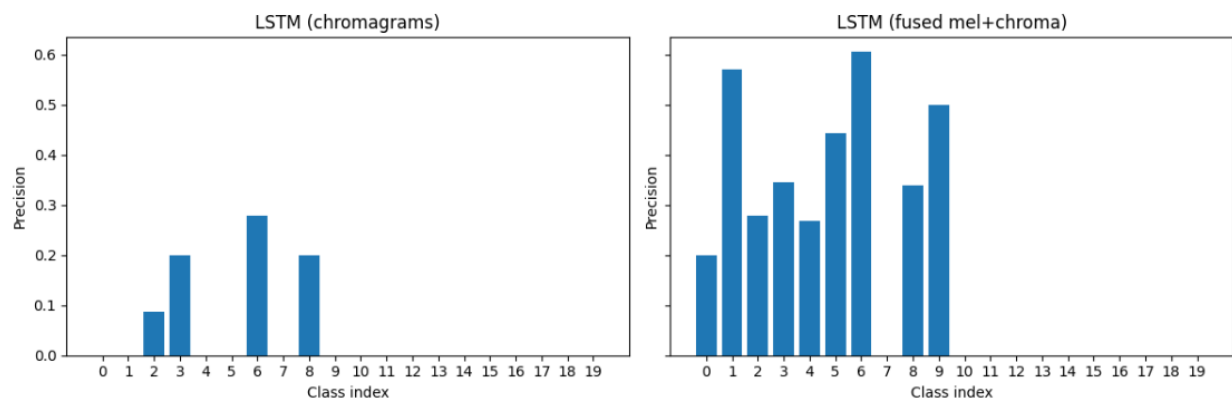
Σε όλες τις περιπτώσεις η αρχική απώλεια είναι περίπου 2.4 και στα mel-based μοντέλα πέφτει γύρω στο 1.7–1.9, δείχνοντας ότι το LSTM μαθαίνει μη τυχαία μοτίβα από τα φασματογράμματα. Το fused (mel+chroma) πετυχαίνει ελαφρώς χαμηλότερο validation loss (περίπου 1.75–1.80), κάτι αναμενόμενο καθώς συνδυάζει φασματική και αρμονική πληροφορία. Αντίθετα, στο καθαρά chroma μοντέλο, το loss μειώνεται μόνο από ~2.46 σε ~2.15, δηλαδή λίγο καλύτερα από τυχαίο, γεγονός που δείχνει ότι με αυτή τη ρύθμιση (LSTM + απλό chroma input) το δίκτυο δεν αξιοποιεί πλήρως την πληροφορία.

■ Βήμα 6: Αξιολόγηση των μοντέλων

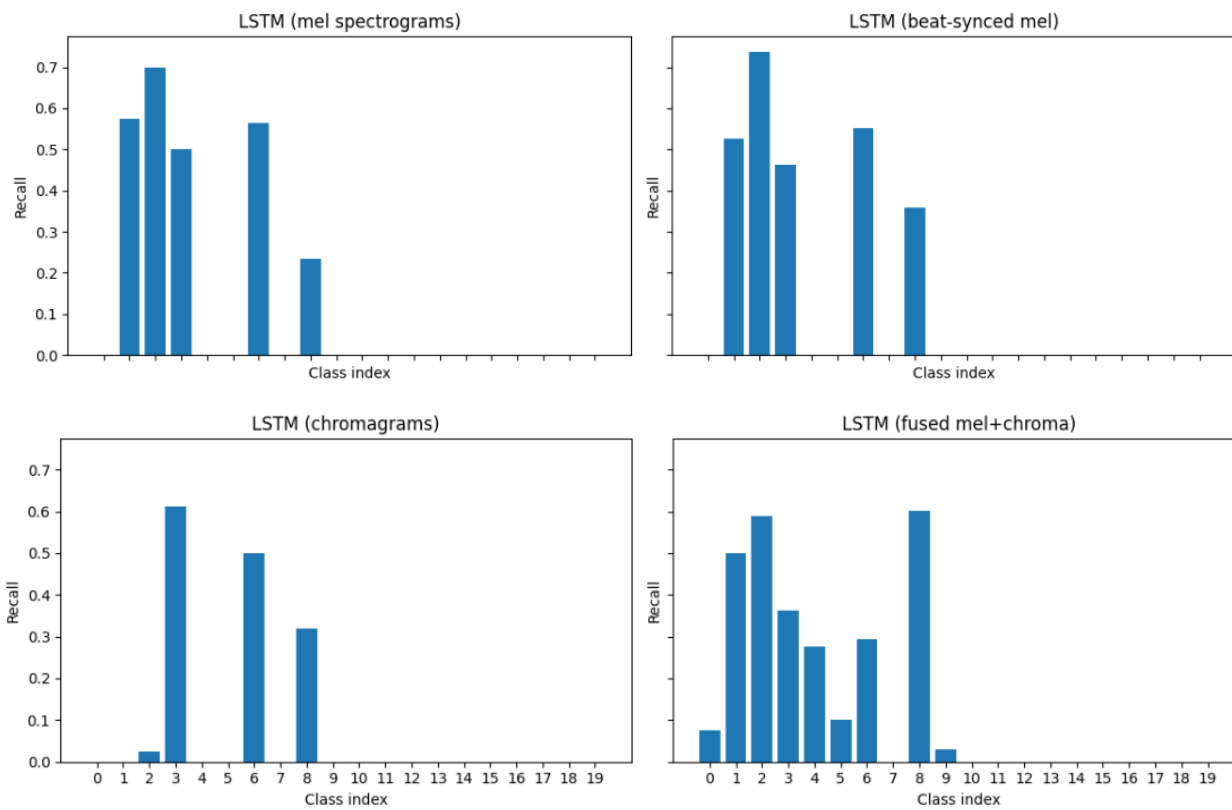
Έπειτα από την εκπαίδευση των μοντέλων υπολογίζουμε τις ζητούμενες μετρικές όπως παρακάτω. Ως disclaimer των αποτελεσμάτων, την ημέρα διεξαγωγής του εργαστηρίου, ο βοηθός μας ενημέρωσε ότι αναμένουμε ποσοστά γύρω στο 30% (όπως και έγινε), επομένως τα αποτελέσματά μας συνάδουν με τις προσδοκίες μας.

α,β,γ,δ) Τα αποτελέσματα φαίνονται ακολούθως για όλες τις μετρικές πάνω στις κλάσεις:

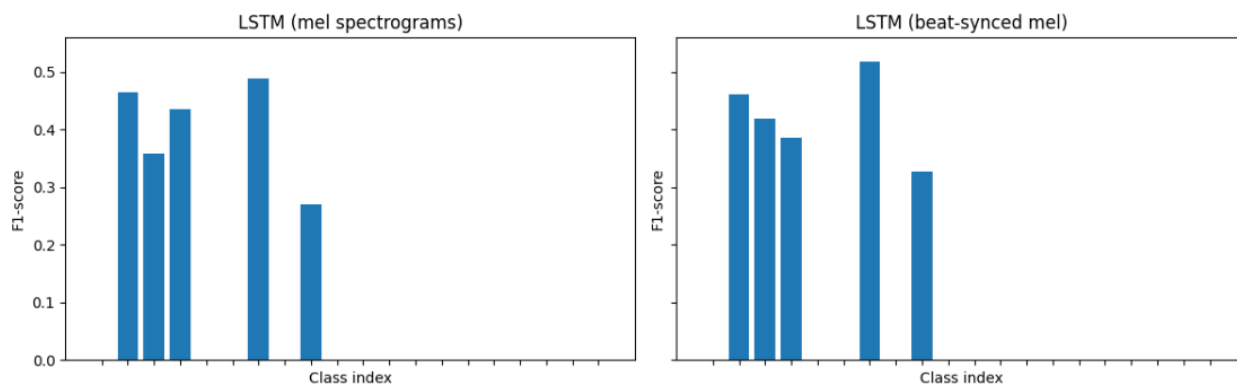


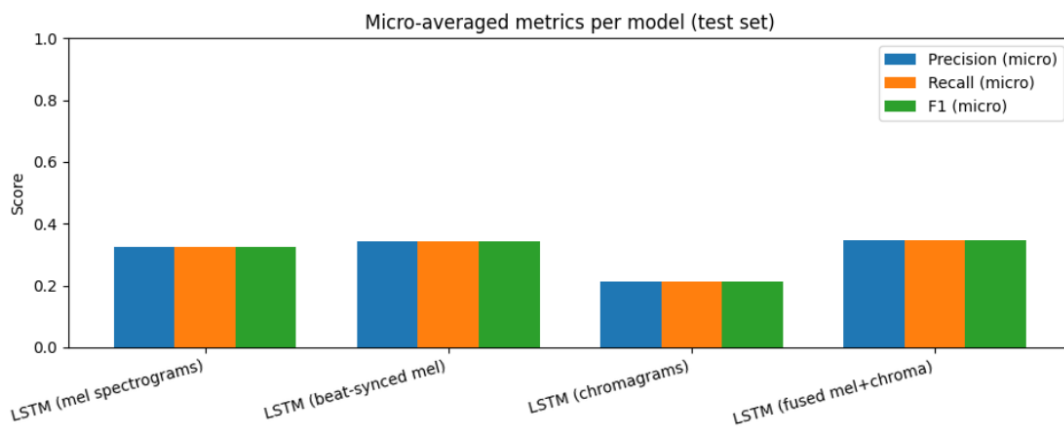
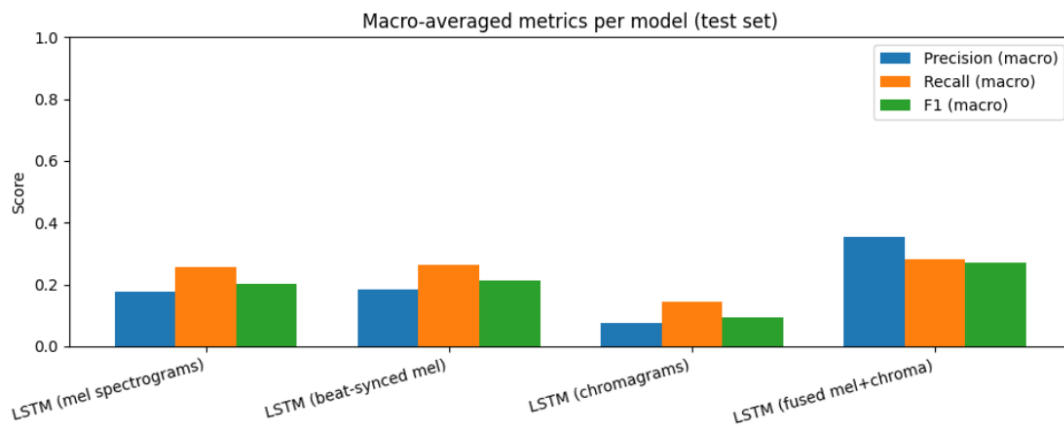
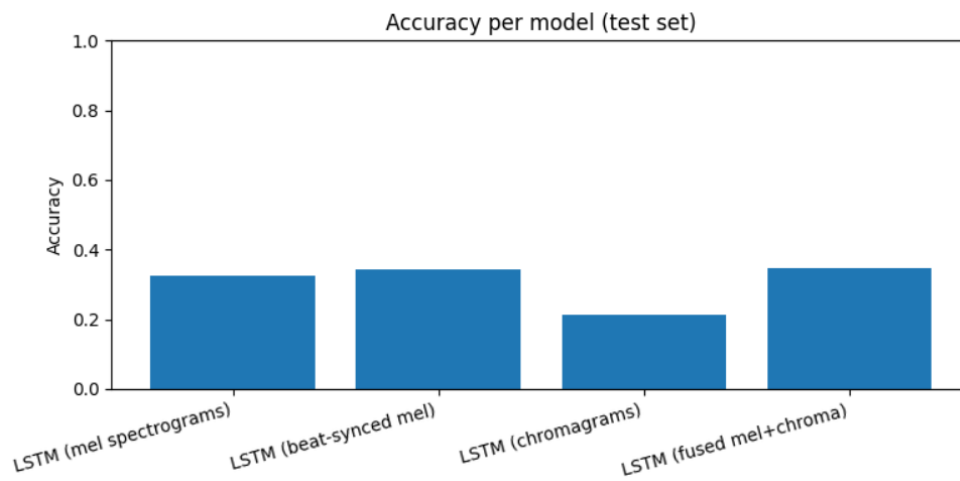
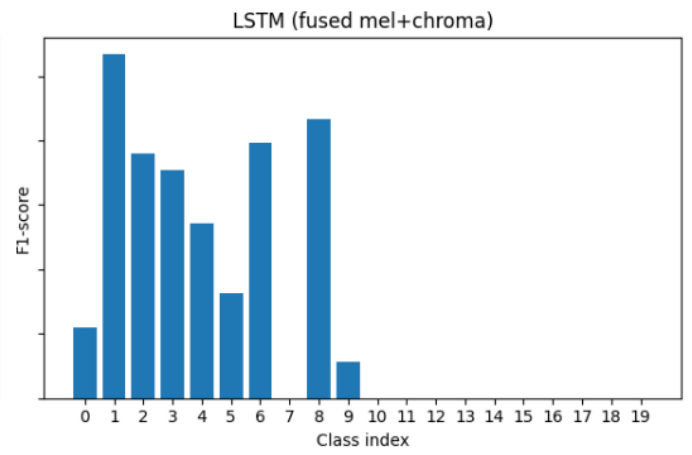
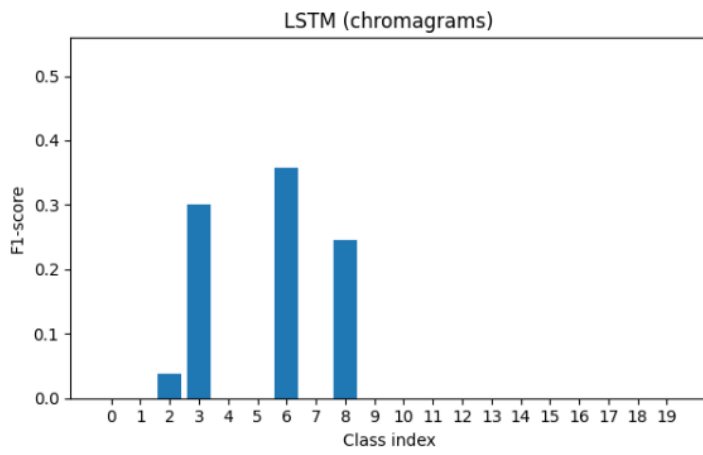


Per-class Recall on test set(s)



Per-class F1-score on test set(s)





Accuracy :

Εκφράζει πόσα δείγματα συνολικά ταξινομήθηκαν σωστά.

$$\text{accuracy} = \frac{\# \text{ σωστών προβλέψεων}}{\# \text{ συνολικών προβλέψεων}}$$

Precision (ανά κλάση):

Εκφράζει το πόσα από τα δείγματα που το μοντέλο ταξινόμησε στην κλάση i ανήκουν πραγματικά στην κλάση i (TP) .

$$\text{precision}_i = \frac{TP_i}{TP_i + FP_i}$$

Υψηλό precision σημαίνει λίγα false positives για αυτή την κλάση.

Recall (ανά κλάση) :

Από όλα τα πραγματικά δείγματα της κλάσης i , πόσα τα βρήκε το μοντέλο;

$$\text{recall}_i = \frac{TP_i}{TP_i + FN_i}$$

Υψηλό recall σημαίνει λίγα false negatives για αυτή την κλάση.

F1-score (ανά κλάση):

Ο συνδυασμός precision & recall, με έμφαση στο πιο χαμηλό από τα δύο είναι το F1-score

$$F1_i = 2 \cdot \frac{\text{precision}_i \cdot \text{recall}_i}{\text{precision}_i + \text{recall}_i}$$

Αν ένα από τα δύο είναι χαμηλό, το F1 πέφτει.

Σε multi-class πρόβλημα, έχουμε ένα precision/recall/F1 για κάθε κλάση το οποίο δεν είναι πρακτικό επομένως τα συνοψίζουμε:

Macro-average :

Δεδομένου πως κάθε κλάση έχει ίσο βάρος υπολογίζουμε το precision, για παράδειγμα, ανά κλάση και μετά:

$$\text{macro-precision} = \frac{1}{K} \sum_{i=1}^K \text{precision}_i$$

(αντίστοιχα για recall, F1).

Micro-average :

Μαζεύουμε **όλα** τα TP, FP, FN από όλες τις κλάσεις μαζί, και μετά υπολογίζουμε precision/recall/F1 σαν να ήταν δυαδικό:

$$\text{micro-precision} = \frac{\sum_i TP_i}{\sum_i (TP_i + FP_i)}, \text{micro-recall} = \frac{\sum_i TP_i}{\sum_i (TP_i + FN_i)}$$

-Στο single-label multi-class, $\text{micro precision} = \text{micro recall} = \text{micro F1} = \text{accuracy}$. Και τα παραπάνω σημαίνουν πως μεγάλες κλάσεις τραβάνε το μέσο όρο προς τα πάνω.

-Το accuracy θα μπορούσε να διαφέρει πολύ από το F1 σε περιπτώσεις που έχουμε ανισοροπία κλάσεων με μεγάλη διαφορά απόδοσης ανα κλάση. Για παράδειγμα 10 κλάσεις, αλλά η μία έχει το 70% των δειγμάτων. Το μοντέλο είναι αποδοτικό στη μεγάλη κλάση αλλά όχι στις μικρές. Το Accuracy μπορεί να είναι αξιοπρεπές όμως το F1 (macro) θα είναι χαμηλό, γιατί πολλές κλάσεις έχουν κακό precision/recall.

-Μεγάλη απόκλιση ανάμεσα σε Micro-Macro F1 έχουμε όταν κάποιες κλάσεις έχουν υψηλό F1 και άλλες χαμηλό. Το μοντέλο ευνοεί τις κλάσεις στις οποίες πετυχαίνει καλά ποσοστά μετρικών.

-Ναι, υπάρχουν προβλήματα όπου το precision είναι πιο βασική μετρική και το αντίστροφο.

Σε κάποια προβλήματα μας νοιάζει περισσότερο το precision: Δηλαδή να είναι όσο γίνεται πιο σωστές οι “θετικές” προβλέψεις (λίγα false positives), ακόμα κι αν χάνουμε κάποια θετικά δείγματα.

Σε άλλα μας νοιάζει περισσότερο το recall: Δηλαδή να μη χάνουμε σχεδόν κανένα πραγματικά θετικό (λίγα false negatives), ακόμα κι αν δεχτούμε περισσότερα λάθος “θετικά”.

-Αυτό σημαίνει ότι προφανώς δεν είναι μία αποδεκτή τιμή F1 αρκετή σε τέτοιες περιπτώσεις.

Στα αποτελέσματά μας βλέπουμε συχνά μεγάλη απόκλιση μεταξύ accuracy/micro-F1 και macro-F1. Αυτό συμβαίνει επειδή το δίκτυο «μαθαίνει» καλύτερα κάποιες κλάσεις (π.χ. κλάσεις 1, 2, 3, 6, 8) και ουσιαστικά αγνοεί άλλες (0, 4, 5, 7, 9), για τις οποίες precision, recall και F1 είναι σχεδόν μηδέν.

Ακόμη, στο δικό μας πρόβλημα, φαίνεται καθαρά στα μοντέλα chroma και mel/beat-mel πως το micro F1 κινείται γύρω στο 0.21–0.34, αλλά το macro F1 είναι σαφώς χαμηλότερο (≈ 0.09 –0.21), λόγω πολλών κλάσεων με μηδενικά metrics.

Όλες οι κλάσεις αντιμετωπίζονται περίπου ισότιμα: δεν υπάρχει genre που να είναι πιο «σημαντικό», μας ενδιαφέρει η συμπεριφορά του μοντέλου σε κάθε είδος και όχι μόνο συνολικά.

Για τους παραπάνω λόγους για το δικό μας πρόβλημα θα διαλέγαμε ως μετρικές:

- macro-F1 (ώστε να αξιολογούμε ισότιμα όλα τα genres), και
- per-class precision/recall/F1 και τα αντίστοιχα διαγράμματα (ώστε να βλέπουμε ποιες κλάσεις αναγνωρίζονται καλά και ποιες αγνοούνται από το δίκτυο).

Με βάση αυτές τις μετρικές, το LSTM με fused mel+chroma έχει τη συνολικά καλύτερη συμπεριφορά (υψηλότερο accuracy/micro-F1 και μεγαλύτερο macro-F1), αλλά τα αποτελέσματα δείχνουν ότι ακόμη και αυτό το μοντέλο αποτυγχάνει σε ορισμένες κλάσεις (π.χ. κλάση 7).

Άρα, η επιλογή «καλύτερου» μοντέλου δεν πρέπει να γίνει μόνο με βάση μία τιμή accuracy ή F1, αλλά με προσεκτική εξέταση των macro metrics και των ανά-κλάση μετρικών, ώστε να είμαστε βέβαιοι ότι η απόδοση είναι ικανοποιητική για όλες τις κλάσεις του προβλήματος.

▪ Βήμα 7.1: 2D CNN

α) Οπτικοποίηση ενός απλού CNN στη σελίδα του Stanford

Στο ερώτημα αυτό δεν υλοποιήσαμε μόνοι μας το συνελκτικό δίκτυο, αλλά χρησιμοποιήσαμε το διαδραστικό demo ConvNetJS στον σύνδεσμο [18]. Το περιβάλλον αυτό μας επιτρέπει να εκπαιδεύσουμε ένα απλό 2D CNN στο σύνολο MNIST και, πολύ σημαντικό για εκπαιδευτικούς σκοπούς, να δούμε αναλυτικά τις ενεργοποιήσεις (activations) και τα διανύσματα βαθμίδων (activation gradients) σε κάθε επίπεδο του δικτύου, καθώς και τα φίλτρα/βάρη που μαθαίνονται.

Περιγραφή του δικτύου και της εκπαίδευσης

Το demo χρησιμοποιεί ως είσοδο εικόνες 28×28 του MNIST, από τις οποίες σε κάθε βήμα εκπαίδευσης «κόβεται» ένα τυχαίο παράθυρο 24×24 . Αυτό λειτουργεί ως απλή μορφή data augmentation, καθώς το δίκτυο βλέπει κάθε φορά ελαφρώς μετατοπισμένες εκδοχές του ίδιου ψηφίου και αποκτά μεγαλύτερη ανθεκτικότητα σε μετατοπίσεις. Το input layer έχει επομένως διαστάσεις $24 \times 24 \times 1$ (γκρι εικόνα).

Η αρχιτεκτονική που χρησιμοποιήσαμε είναι η προεπιλεγμένη του demo και αποτελείται από:

- **Επίπεδο εισόδου (input):** $24 \times 24 \times 1$.
- **1ο συνελκτικό επίπεδο (conv):** 8 φίλτρα $5 \times 5 \times 1$, stride 1. Η έξοδος έχει διαστάσεις $24 \times 24 \times 8$.
- **ReLU:** εφαρμογή της μη γραμμικής συνάρτησης ενεργοποίησης ReLU πάνω στις ενεργοποιήσεις του 1ου conv.
- **1ο επίπεδο υποδειγματοληψίας (pool):** max-pooling 2×2 με stride 2, που μειώνει τις διαστάσεις σε $12 \times 12 \times 8$.
- **2ο συνελκτικό επίπεδο (conv):** 16 φίλτρα 5×5 που «βλέπουν» και τα 8 κανάλια εισόδου ($5 \times 5 \times 8$). Η έξοδος είναι $12 \times 12 \times 16$.
- **ReLU:** και πάλι μη γραμμικότητα πάνω στις ενεργοποιήσεις του δεύτερου conv.
- **2ο επίπεδο pool:** max-pooling 3×3 με stride 3, που μειώνει τις διαστάσεις του χάρτη χαρακτηριστικών σε $4 \times 4 \times 16$.
- **Πλήρως συνδεδεμένο επίπεδο (fc):** λαμβάνει ως είσοδο το επίπεδο $4 \times 4 \times 16$ (256 νευρώνες) και το χαρτογραφεί σε 10 logits, ένα για κάθε κλάση ψηφίου (0–9).
- **Softmax:** μετατρέπει τα logits σε πιθανότητες κλάσεων.

Για την εκπαίδευση χρησιμοποιείται ο αλγόριθμος Adadelta (adaptive βήμα εκπαίδευσης), με τις παραμέτρους που φαίνονται στο interface: learning rate περίπου $1e-4$, momentum 0.9, batch size 64 και weight decay 0.001, καθώς αυτές οι παράμετροι είναι ιδιαίτερα αποδοτικές (ύστερα από σχετική αναζήτηση στο internet). Καθώς αφήνουμε το μοντέλο να εκπαιδευτεί για αρκετές επαναλήψεις (το αφήσαμε αρκετά λεπτά, για πάνω από 40,000 δείγματα, παρόλο που το site έλεγε να το αφήσουμε λίγα λεπτά μόλις), βλέπουμε ότι:

- Το classification loss πέφτει γρήγορα από μια αρχική υψηλή τιμή σε περίπου 0.13, ενώ στα ενδιάμεσα στάδια είχε φτάσει και 0.01

- Το L2 weight decay loss παραμένει μικρό (της τάξης 0.003–0.004), δρώντας κυρίως ως όρος κανονικοποίησης.
- Το training accuracy φτάνει περίπου στο 95% (είχε φτάσει και 99%), ενώ το **validation accuracy** αγγίζει το 99%, όπως είναι αναμενόμενο για MNIST με ένα σχετικά ικανό CNN.
- Στο γράφημα του loss βλέπουμε ότι η απώλεια μειώνεται απότομα στην αρχή και στη συνέχεια σταθεροποιείται σε χαμηλή τιμή, με μικρές διακυμάνσεις γύρω από το ελάχιστο που έχει διαμορφώσει, καθώς ο αλγόριθμος συνεχίζει να βελτιστοποιεί τα βάρη.

Αυτά τα νούμερα επιβεβαιώνουν ότι το δίκτυο έχει μάθει πολύ καλά το πρόβλημα ταξινόμησης ψηφίων, χωρίς να φαίνεται να υπερπροσαρμόζεται υπερβολικά στο train σε σχέση με το validation (το validation accuracy παραμένει πολύ υψηλό).

Η πιο ενδιαφέρουσα πτυχή του demo είναι η οπτικοποίηση των ενεργοποιήσεων (activations) ανά επίπεδο, καθώς και των gradients και των weights. Αυτό μας επιτρέπει να δούμε στην πράξη την ιεραρχία χαρακτηριστικών που μαθαίνει ένα CNN. Ειδικότερα, παραθέτουμε τα ακόλουθα screenshots, όπως ζητείται:

ConvNetJS MNIST demo

Description

This demo trains a Convolutional Neural Network on the [MNIST digits dataset](#) in your browser, with nothing but Javascript. The dataset is fairly easy and one should expect to get somewhere around 99% accuracy within few minutes. I used [this python script](#) to parse the [original files](#) into batches of images that can be easily loaded into page DOM with img tags.

This network takes a 28x28 MNIST image and crops a random 24x24 window before training on it (this technique is called data augmentation and improves generalization). Similarly to do prediction, 4 random crops are sampled and the probabilities across all crops are averaged to produce final predictions. The network runs at about 5ms for both forward and backward pass on my reasonably decent Ubuntu+Chrome machine.

By default, in this demo we're using Adadelta which is one of per-parameter adaptive step size methods, so we don't have to worry about changing learning rates or momentum over time. However, I still included the text fields for changing these if you'd like to play around with SGD+Momentum trainer.

Report questions/bugs/suggestions to [@karpathy](#).

Training Stats

resume

Forward time per example: 2ms

Backprop time per example: 4ms

Classification loss: 0.13762

L2 Weight decay loss: 0.00351

Training accuracy: 0.95

Validation accuracy: 0.99

Examples seen: 43207

Learning rate:

change

Momentum:

change

Batch size:

change

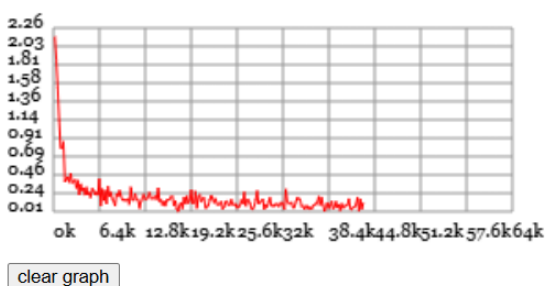
Weight decay:

change

save network snapshot as JSON

init network from JSON snapshot

Loss:




```

layer_defs = [];
layer_defs.push({type:'input', out_sx:24, out_sy:24, out_depth:1});
layer_defs.push({type:'conv', sx:5, filters:8, stride:1, pad:2, activation:'relu'});
layer_defs.push({type:'pool', sx:2, stride:2});
layer_defs.push({type:'conv', sx:5, filters:16, stride:1, pad:2, activation:'relu'});
layer_defs.push({type:'pool', sx:3, stride:3});
layer_defs.push({type:'softmax', num_classes:10});

net = new convnetjs.Net();
net.makeLayers(layer_defs);

trainer = new convnetjs.SGDTrainer(net, {method:'adadelta', batch_size:20, l2_decay:0.001});

```

change network

Network Visualization

input (24x24x1)
max activation: 1, min: 0
max gradient: 0.00002, min: -0.00004

Activations:



Activation Gradients:



conv (24x24x8)
filter size 5x5x1, stride 1
max activation: 3.66141, min: -3.30518
max gradient: 0.00001, min: -0.00002
parameters: $8 \times 5 \times 5 \times 1 + 8 = 208$

Activations:



Activation Gradients:



Weights:

()()()()()()()(

Weight Gradients:

()()()()()()()(

relu (24x24x8)
max activation: 3.66141, min: 0
max gradient: 0.00001, min: -0.00002

Activations:



Activation Gradients:



pool (12x12x8)
pooling size 2x2, stride 2
max activation: 3.66141, min: 0
max gradient: 0.00001, min: -0.00002

Activations:



Activation Gradients:



conv (12x12x16)
filter size 5x5x8, stride 1
max activation: 6.33837, min: -13.27822
max gradient: 0, min: -0.00002
parameters: $16 \times 5 \times 5 \times 8 + 16 = 3216$

Activations:



Activation Gradients:



Weights:

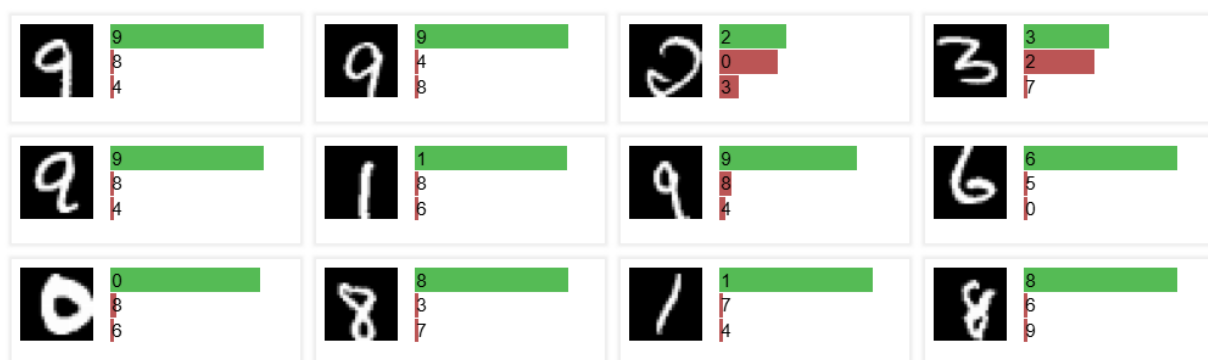
()()()()()()()()()()()()()()()(

Weight Gradients:

()()()()()()()()()()()()()()()(

relu (12x12x16) max activation: 6.33837, min: 0 max gradient: 0, min: -0.00002	Activations:  Activation Gradients: 
pool (4x4x16) pooling size 3x3, stride 3 max activation: 6.33837, min: 0 max gradient: 0, min: -0.00002	Activations:  Activation Gradients: 
fc (1x1x10) max activation: 2.19726, min: -22.93499 max gradient: 0.00001, min: -0.00003 parameters: 10x256+10 = 2570	Activations:  Activation Gradients: 
softmax (1x1x10) max activation: 0.99997, min: 0 max gradient: 0, min: 0	Activations: 

Example predictions on Test set



1. Είσοδος (input 24×24×1) :

Εδώ βλέπουμε απλώς το πραγματικό ψηφίο που τροφοδοτείται στο δίκτυο. Οι βαθμίδες (activation gradients) στο input δείχνουν ποιες περιοχές της εικόνας συνεισφέρουν περισσότερο στο σφάλμα – συνήθως τα pixels πάνω στις γραμμές/πινελιές του ψηφίου έχουν τις μεγαλύτερες τιμές gradient.

2. 1ο συνελκτικό επίπεδο (conv 24×24×8):

Στις ενεργοποιήσεις αυτού του επιπέδου βλέπουμε 8 διαφορετικούς χάρτες, καθένας από τους οποίους μοιάζει με «φιλτραρισμένη» εκδοχή του αρχικού ψηφίου. Τα φίλτρα που εμφανίζονται στο πεδίο «Weights» έχουν την μορφή μικρών 5×5 «patches» που αντιστοιχούν σε τοπικούς ανιχνευτές χαρακτηριστικών, όπως:

- ανίχνευση κάθετων ή οριζόντιων γραμμών
- διαγώνια strokes
- απλά blobs φωτεινότητας.

Οι αντίστοιχες ενεργοποιήσεις τονίζουν τα μέρη του ψηφίου όπου το συγκεκριμένο φίλτρο «ταιριάζει» καλύτερα.

3. ReLU (24×24×8):

Μετά το ReLU, οι χάρτες ενεργοποιήσεων γίνονται πιο αραιοί: πολλές τιμές μηδενίζονται, και απομένουν μόνο οι «δυνατές» αποκρίσεις. Αυτό βοηθά στο να κρατηθούν μόνο οι σημαντικές

ανιχνεύσεις χαρακτηριστικών, ενώ ταυτόχρονα εισάγεται μη γραμμικότητα ώστε το δίκτυο να μπορεί να μάθει πιο περίπλοκες συναρτήσεις.

4. **1o Pooling (12×12×8):**

Το max-pooling 2×2 μειώνει την ανάλυση στο μισό (από 24×24 σε 12×12), κρατώντας σε κάθε μικρή περιοχή μόνο τη μέγιστη τιμή. Στις οπτικοποιήσεις φαίνεται ότι τα ψηφία γίνονται πιο «παχιά», αλλά η βασική μορφή τους διατηρείται.

5. **2o συνελκτικό επίπεδο (conv 12×12×16):**

Το δεύτερο conv επίπεδο έχει 16 φίλτρα 5×5×8, δηλαδή κάθε φίλτρο «βλέπει» και όλους τους 8 χάρτες χαρακτηριστικών από το προηγούμενο επίπεδο. Οι οπτικοποιήσεις δείχνουν πιο σύνθετα patterns, όπως:

- καμπύλες, γωνίες και συνδυασμούς strokes,
- τμήματα που μοιάζουν με «κομμάτια» ολόκληρων ψηφίων (π.χ. το άνω ημικύκλιο του 9 ή η κάτω καμπύλη του 5).

6. **ReLU και 2o Pooling (4×4×16):**

Μετά το δεύτερο ReLU και το pooling 3×3 (stride 3), οι χάρτες συρρικνώνονται σε 4×4×16. Στο σημείο αυτό, η χωρική ανάλυση είναι πολύ μικρή και κάθε νευρώνας αντιστοιχεί σε μια σχετικά μεγάλη περιοχή του αρχικού input. Οι ενεργοποιήσεις έχουν πλέον περισσότερο τον ρόλο ενεργοποίησης συγκεκριμένων ψηφίων-προτύπων.

7. **Πλήρως συνδεδεμένο και softmax (1×1×10):**

Το flatten επίπεδο 4×4×16=256 συνδέεται σε 10 εξόδους. Στις ενεργοποιήσεις του fully connected layer βλέπουμε ένα διάνυσμα 10 τιμών (logits) και στο softmax ένα αντίστοιχο διάνυσμα πιθανοτήτων. Τα παραδείγματα προβλέψεων στο τέλος του demo δείχνουν ότι:

- η σωστή κλάση έχει πιθανότητα πολύ κοντά στο 1 (μεγάλες πράσινες μπάρες),
- οι υπόλοιπες κλάσεις έχουν πολύ μικρές πιθανότητες (κόκκινες μπάρες).

Αυτό επιβεβαιώνει ότι το CNN έχει μάθει πολύ διαχωριστικές αναπαραστάσεις: τα features των τελευταίων επιπέδων είναι σχεδόν ξεχωριστά για κάθε ψηφίο.

Συμπεράσματα, τι μαθαίνει το δίκτυο:

Μέσα από το ConvNetJS demo, βλέπουμε στην πράξη τη βασική αρχή των 2D CNNs:

- Τα πρώτα επίπεδα μαθαίνουν τοπικά, χαμηλού επιπέδου χαρακτηριστικά (γραμμές, άκρες, μικρές γωνίες).
- Τα ενδιάμεσα επίπεδα συνδυάζουν αυτά τα χαρακτηριστικά σε πιο σύνθετα μοτίβα (μέρη ψηφίων).
- Τα τελευταία επίπεδα λειτουργούν ως ταξινομητές πάνω σε υψηλού επιπέδου χαρακτηριστικά, που αντιστοιχούν ουσιαστικά σε «πρότυπα ψηφίων».

β) Υλοποίηση 2D CNN για τα φασματογραφήματα Mel

Ακολουθώντας, τον βοηθητικό κώδικα που μας δίνεται υλοποιήθηκε 2D CNN. Το input του δικτύου είναι τα mel φασματογραφήματα του Βήματος 4–5, τα οποία θεωρούμε ως «εικόνες» 1×128×T (1 κανάλι, 128 mel ζώνες και T χρονικά frames, μετά από cropping/padding στο σταθερό μήκος MAX_LENGTH).

Ο κορμός του δικτύου υλοποιείται στην κλάση CNNBackbone, η οποία αποτελείται από τέσσερα συνελκτικά blocks με αριθμό φίλτρων:

- Conv block 1: 32 φίλτρα 5×5, batch normalization, ReLU, max-pooling 2×2.
- Conv block 2: 64 φίλτρα 5×5, batch normalization, ReLU, pooling 2×2.
- Conv block 3: 128 φίλτρα 3×3, batch normalization, ReLU, pooling 2×2.
- Conv block 4: 256 φίλτρα 3×3, batch normalization, ReLU, pooling 2×2.

Για να επαναχρησιμοποιήσουμε τον ίδιο κώδικα εκπαίδευσης, προσθέσαμε τον CNNBackbone μέσα στην ήδη υπάρχουσα κλάση Classifier (NUM_CATEGORIES, backbone), η οποία προσθέτει ένα τελικό γραμμικό επίπεδο προς τις 10 κατηγορίες, εφαρμόζει τη softmax και χρησιμοποιεί nn.CrossEntropyLoss όπως και τα LSTM του Βήματος 5–6.

Ολόκληρο το μοντέλο μεταφέρεται σε GPU (DEVICE='cuda') και εκπαιδεύεται με Adam optimizer σε learning rate της τάξης του 10^{-4} , ίδιο BATCH_SIZE με πριν και early stopping (ίδιος μηχανισμός με το Βήμα 5).

Έτσι, η μόνη ουσιαστική διαφορά σε σχέση με τα προηγούμενα πειράματα είναι ο backbone (LSTM vs 2D CNN), ενώ όλα τα υπόλοιπα στοιχεία του pipeline (datasets, loaders, loss, scheduler/early stopping) παραμένουν κοινά.

γ) Επιλογή αρχιτεκτονικής και υπερπαραμέτρων

Για την επιλογή υπερπαραμέτρων ακολουθήσαμε τις υποδείξεις της εκφώνησης, αλλά και τις γενικές «καλές πρακτικές» για CNNs σε φασματογραφήματα (έπειτα από αναζήτηση στο διαδίκτυο):

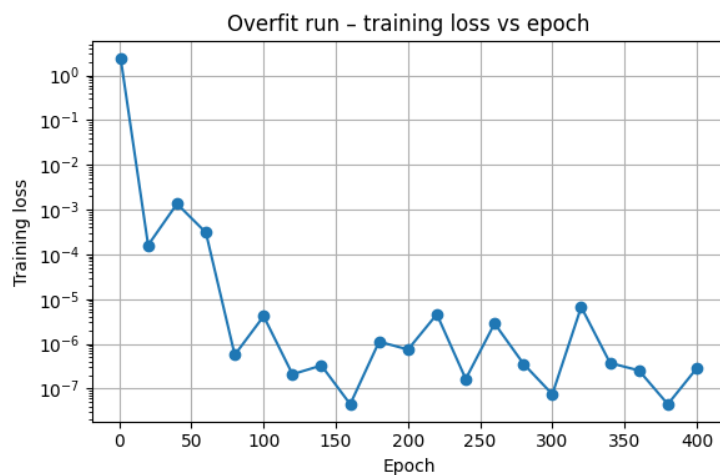
- Χρησιμοποιήσαμε 4 συνελκτικά επίπεδα με φίλτρα [32, 64, 128, 256], καθώς αυτή η κλιμάκωση (διπλασιασμός του πλήθους φίλτρων ανά επίπεδο) εμφανίζεται συχνά στη βιβλιογραφία (VGG-style δίκτυα) και επιτρέπει στο μοντέλο να μαθαίνει από απλά τοπικά patterns σε πιο σύνθετες δομές χωρίς να εκτοξεύεται ο αριθμός παραμέτρων.
- Τα μεγέθη των πυρήνων 5×5 στα πρώτα επίπεδα και 3×3 στα βαθύτερα αφήνουν στο δίκτυο αρκετή ευελιξία να «βλέπει» τοπικές δομές χρόνου–συχνότητας (π.χ. μικρά ρυθμικά/αρμονικά μοτίβα), ενώ τα max-pooling layers μειώνουν σταδιακά τη χωρική ανάλυση και κρατούν μόνο τις πιο ισχυρές αποκρίσεις, όπως προτείνεται και στο παράδειγμα του ConvNetJS στο MNIST.
- Το feature_size = 1024 (στον κώδικα που μας δίνεται ήταν 1000, αλλά το κάναμε δύναμη του 2) στο fully connected επίπεδο αποτελεί έναν συμβιβασμό: είναι αρκετά μεγάλο για να κωδικοποιεί πλούσια χαρακτηριστικά για τα 10 genres, αλλά όχι υπερβολικά μεγάλο ώστε να οδηγεί αναγκαστικά σε overfitting ή σε τεράστιο αριθμό παραμέτρων.
- Για τη βελτιστοποίηση χρησιμοποιήσαμε Adam με learning rate της τάξης του 10^{-4} , όπως συχνά προτείνεται για CNNs σε audio και εικόνες, και βάλαμε early stopping με patience 5 εποχών, ώστε να σταματά η εκπαίδευση όταν το validation loss σταματήσει να βελτιώνεται.
- Διατηρήσαμε το ίδιο train/validation split και τον ίδιο BATCH_SIZE με τα LSTM του Βήματος 5, ώστε η σύγκριση των αποτελεσμάτων (ιδίως στο Βήμα 6 και εδώ) να είναι δίκαιη.

Συνολικά, η αρχιτεκτονική είναι σχετικά «βαθιά» για τα δεδομένα του MNIST-style φασματογραφήματος, αλλά παραμένει διαχειρίσιμη σε χρόνο/μνήμη, επιτρέποντάς μας να κάνουμε 40 epochs (με early stopping γύρω στην 18η) σε λογικό χρόνο στο Kaggle GPU.

δ) Πείραμα υπερεκπαίδευσης (overfitting σε ένα batch)

Για να ελέγξουμε ότι ο κώδικας του CNN λειτουργεί σωστά, εκτελέσαμε ένα πείραμα υπερεκπαίδευσης αντίστοιχο με αυτό του LSTM στο Βήμα 5: καλέσαμε τη συνάρτηση train με overfit_batch=True και αφήσαμε το δίκτυο να εκπαιδευτεί μόνο πάνω σε ένα (ή σε λίγα) batches για 400 εποχές.

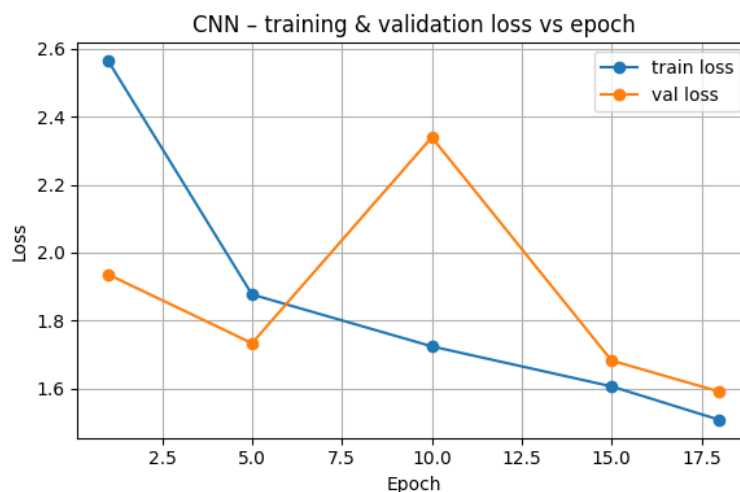
Αυτά, φαίνονται και ακολούθως (διάγραμμα μόνο για αναφορά):



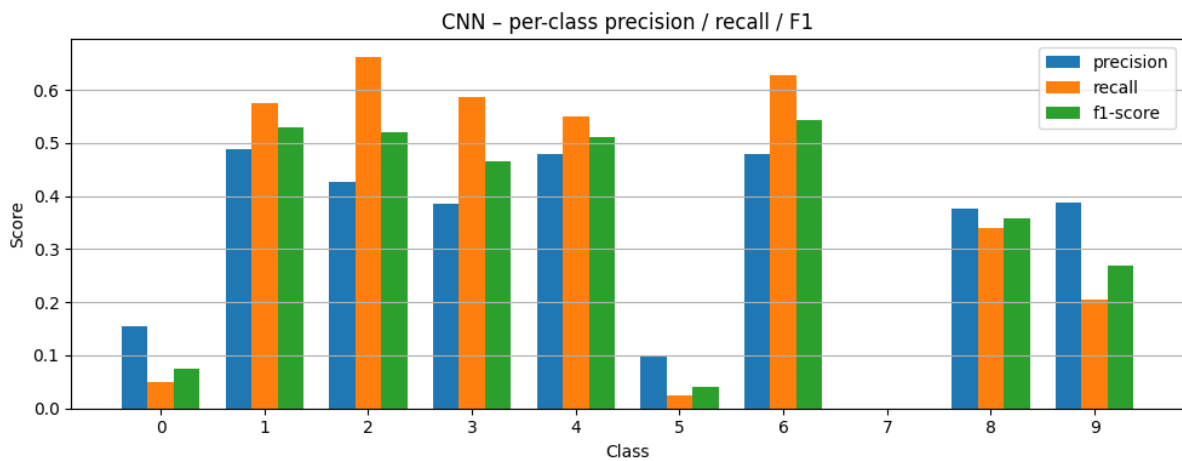
Οι απώλειες τείνουν στο μηδέν, υπερεκπαίδευση. Ομοίως με τα προηγούμενα ερωτήματα βλέπουμε πως το `overfit_batch` δουλεύει επαρκώς.

ε) Κανονική εκπαίδευση, αξιολόγηση και σύγκριση με LSTM

Τέλος, εκπαιδύσαμε το ίδιο CNN κανονικά (`overfit_batch=False`) στο πλήρες train set των mel φασματογραφημάτων για έως και 40 εποχές. Με τη χρήση early stopping, η εκπαίδευση σταμάτησε γύρω στην 18η εποχή: το training loss συνέχισε να μειώνεται (π.χ. από 2.56 στην 1η εποχή σε 1.50 στην 18η), αλλά το validation loss έπαυε να βελτιώνεται μετά την 10η-11η εποχή και στη συνέχεια άρχισε να αυξάνεται (ένδειξη υπερεκπαίδευσης). Τα αποτελέσματα της εκπαίδευσης και στο test set φαίνονται ακολούθως:



Στη συνέχεια αξιολογήσαμε το εκπαιδευμένο μοντέλο στο test set των mel φασματογραφημάτων, υπολογίζοντας τις ίδιες μετρικές με το Βήμα 6:

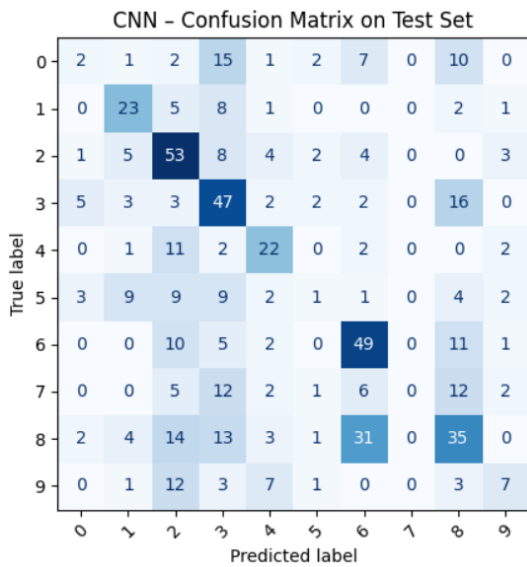


- **Accuracy (επιτυχία):** 0.4157, δηλαδή περίπου 41.6% σωστές προβλέψεις σε 10 κλάσεις.
- **Micro-averaged precision/recall/F1:** όλα ίσα με 0.4157 (όπως αναμενόταν, αφού στο πολυταξικό setting ενός label ανά δείγμα το micro-F1 ταυτίζεται με το accuracy).
- **Macro-averaged precision/recall/F1:**
 - Precision_macro ≈ 0.3280
 - Recall_macro ≈ 0.3624
 - F1_macro ≈ 0.3312

Επιπλέον, το classification_report δείχνει αναλυτικά τις ανά-κλάση τιμές:

- Καλύτερα αποτελέσματα σε κλάσεις όπου το μοντέλο έχει αρκετά σωστά hits, π.χ.
 - κλάση 1: F1 ≈ 0.53
 - κλάση 2: F1 ≈ 0.52
 - κλάση 3: F1 ≈ 0.47
 - κλάση 4: F1 ≈ 0.51
 - κλάση 6: F1 ≈ 0.54
- Χαμηλή απόδοση σε πιο «δύσκολες» κλάσεις, π.χ.
 - κλάση 0: F1 ≈ 0.08
 - κλάση 5: F1 ≈ 0.04
 - κλάση 7: F1 ≈ 0.00 (δεν υπάρχει ούτε μία σωστή πρόβλεψη).
 - κλάση 9: F1 ≈ 0.27 .

Το αντίστοιχο confusion matrix επιβεβαιώνει αυτά τα συμπεράσματα:



Το δίκτυο αναγνωρίζει αρκετά καλά συγκεκριμένα είδη (τα οποία αντιστοιχούν στις κλάσεις 1,2,3,4,6,8), αλλά σε άλλες κλάσεις τείνει να «μπερδεύεται» συστηματικά με τα ίδια 1–2 genres (π.χ. οι πραγματικές κλάσεις 5 και 7 προβλέπονται συχνά ως κλάση 2, 3 ή 8). Σε αντίθεση με τα αποτελέσματα του Βήματος 6, όμως, δεν έχουμε πλέον κλάσεις με μηδενική παρουσία στο test set. Όλες οι κλάσεις εμφανίζονται στο test και το μοντέλο παράγει προβλέψεις για καθεμία από αυτές, έστω κι αν σε ορισμένες δεν πετυχαίνει σωστά παραδείγματα.

- Το καλύτερο LSTM (fused mel+chroma) είχε accuracy/micro-F1 ≈ 0.348 και macro-F1 ≈ 0.27 .
- Το 2D CNN πάνω στα mel φασματογραφήματα φτάνει σε accuracy ≈ 0.416 και macro-F1 ≈ 0.331 την ισορροπία μεταξύ των κλάσεων.

- Παρότι ορισμένες κλάσεις παραμένουν προβληματικές (π.χ. η κλάση 7 με F1 ≈ 0), η διαφορά μεταξύ micro και macro F1 είναι μικρότερη σε σχέση με το chroma LSTM, κάτι που σημαίνει ότι το CNN είναι συνολικά πιο ομοιόμορφο και δεν αγνοεί εντελώς συγκεκριμένες κλάσεις, αλλά έχει πιο ήπια πτώση απόδοσης εκεί.

Βήμα 7.2: AST - Audio Spectrogram Transformer

α) Το μοντέλο AST ένα καθαρά attention-based δίκτυο για ταξινόμηση ήχου, χωρίς συνελκτικά επίπεδα. Αρχικά, το ηχητικό σήμα μετατρέπεται σε φασματογράφημα log-Mel 128 φίλτρων, από το οποίο δημιουργούνται δισδιάστατα patches 16×16 με επικάλυψη στον χρόνο και στη συχνότητα. Κάθε patch προβάλλεται γραμμικά σε ένα διάνυσμα 768 διαστάσεων, στο οποίο προστίθεται μια positional embedding ώστε να κωδικοποιείται η χωρική δομή του φασματογράμματος.

Επιπλέον, προσαρτάται ένα ειδικό token CLS στην αρχή της ακολουθίας. Η ακολουθία αυτή τροφοδοτείται σε έναν standard Transformer encoder (12 στρώματα, 12 heads), και το τελικό embedding του CLS χρησιμοποιείται ως συνολική αναπαράσταση του σήματος ήχου. Τέλος, έχουμε μία σιγμοειδή συνάρτηση πριν την έξοδο προς ταξινόμηση.

Για να αντιμετωπιστεί η ανάγκη για μεγάλα σύνολα εκπαίδευσης, το AST αξιοποιεί μεταφορά μάθησης από Vision Transformers προεκπαιδευμένα στο ImageNet: επαναχρησιμοποιεί τα βάρη του patch embedding και του Transformer encoder, προσαρμόζοντας μόνο την είσοδο και την positional embedding σε διαφορετικές χωρικές διαστάσεις.

Με αυτόν τον τρόπο, το μοντέλο μπορεί να επωφεληθεί από γνώση που έχει μάθει σε οπτικά δεδομένα, ενώ εφαρμόζεται απευθείας σε μεταβλητού μήκους audio spectrograms.

β)

Υλοποίηση

Χρησιμοποιούμε τα mel-spectrograms ως εικόνες (άξονες: χρόνος $\times$ συχνότητα). Αντί για συνελκτικά φίλτρα όπως στο 2D CNN, το AST χωρίζει το spectrogram σε μικρά patches 16×16 και τα περνά από ένα Vision

Transformer τύπου DeiT. Με το self-attention, το μοντέλο μπορεί να «βλέπει» ταυτόχρονα όλες τις χρονικές στιγμές και όλες τις συχνότητες.

Χρησιμοποιήθηκε αυτούσιος ο κώδικας ASTBackbone που δόθηκε στο εργαστήριο, προσαρμοσμένος στη βιβλιοθήκη timm της Kaggle. Επιλέξαμε το προεκπαιδευμένο μοντέλο `deit_base_distilled_patch16_384` σε ImageNet και το προσαρμόσαμε σε μονοκάναλα mel-spectrograms με διαστάσεις περίπου $T \times 128$.

Ορίσαμε `strides fstride = tstride = 10` ώστε τα patches να είναι ελαφρώς επικαλυπτόμενα σε χρόνο και συχνότητα, όπως προτείνεται στο σχετικό paper/υποδείξεις. Πάνω από το backbone προσθέσαμε ένα MLP head που παράγει διανύσματα χαρακτηριστικών μεγέθους 1024, ώστε να είναι συγκρίσιμο με το 2D CNN.

Χρησιμοποιήσαμε τον γενικό ταξινομητή Classifier του εργαστηρίου και Adam optimizer με learning rate $1e-4$, ίδιο με τα LSTM/CNN. Η εκπαίδευση έγινε στα ίδια train/validation/test splits με τα βήματα 5–7.1. Η αξιολόγηση έγινε με τα ίδια metrics όπως στο Βήμα 6 και 7.1 (accuracy, micro/macro precision/recall/F1, classification_report, confusion matrix), ώστε τα αποτελέσματα να είναι πλήρως συγκρίσιμα.

Πείραμα overfitting

Για να ελέγξουμε ότι η υλοποίηση είναι σωστή, εκπαιδεύσαμε το μοντέλο `ast_model_overfit` σε ένα πολύ μικρό υποσύνολο του train set, με `overfit_batch=True` και 400 epochs. Η απώλεια στο training set ξεκινά από περίπου 2.25 στην 1η εποχή και καταλήγει σε περίπου $2.25 \cdot 10^{-5}$ στην 400ή. Το πείραμα συνεπώς είναι πετυχημένο.

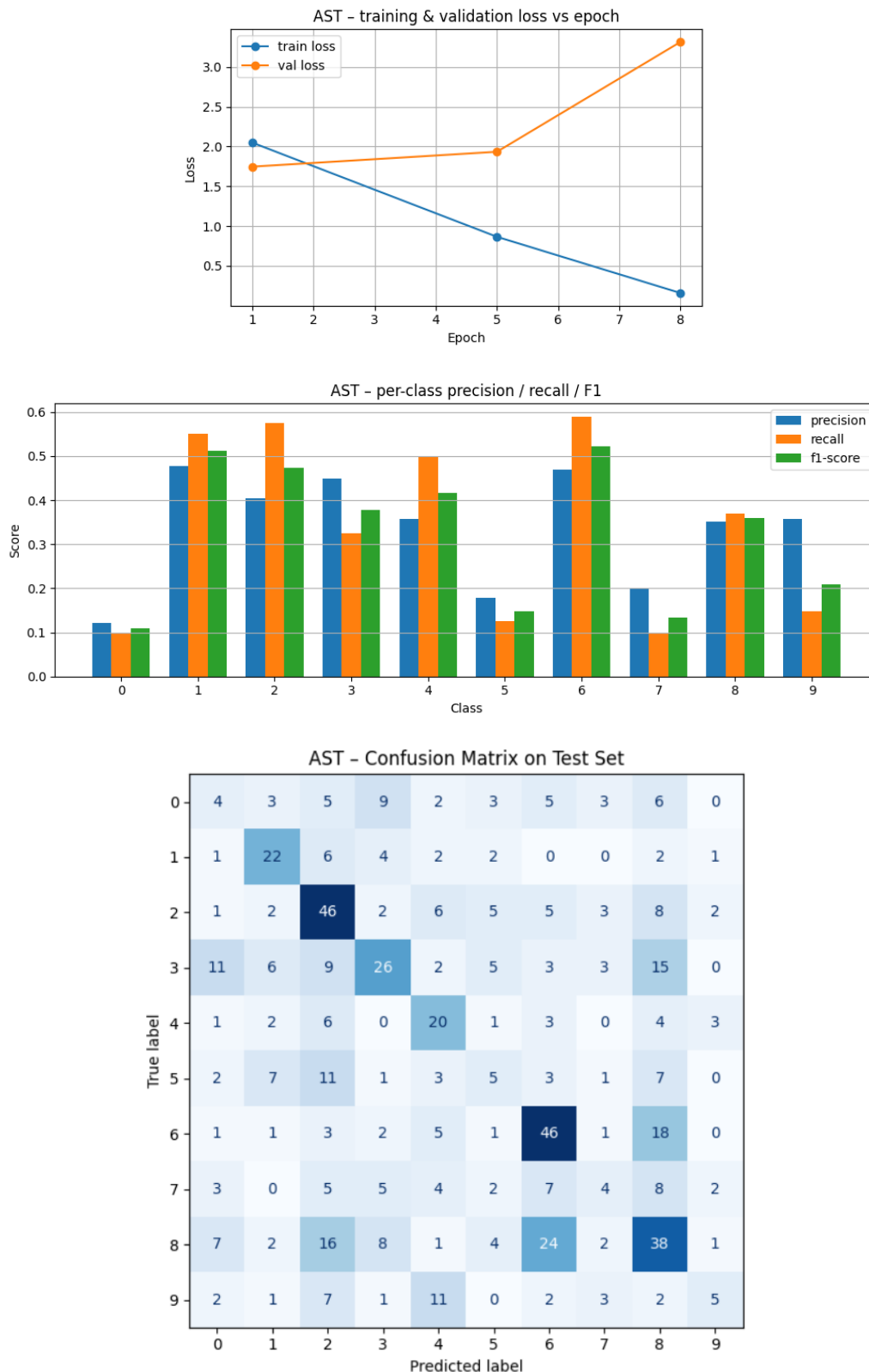
Κανονική εκπαίδευση και αποτελέσματα στο test set

Εκπαιδεύσαμε το κύριο `ast_model` στο πλήρες train set για έως 40 epochs με early stopping. Η εκπαίδευση σταματά στο 8ο epoch, καθώς η validation loss αρχίζει να αυξάνεται. Στο test set παίρνουμε:

- Accuracy ≈ 0.3757
- Micro-avg P/R/F1 $\approx 0.3757 / 0.3757 / 0.3757$
- Macro-avg P/R/F1 $\approx 0.3365 / 0.3381 / 0.3261$

Η διαφορά micro–macro δείχνει ότι το AST αποδίδει ανόμοια μεταξύ κλάσεων: ο micro-μέσος όρος είναι υψηλότερος από τον macro-μέσο όρο, άρα το μοντέλο τα πάει καλύτερα σε ορισμένες κλάσεις και χειρότερα σε άλλες.

Τα αποτελέσματα φαίνονται αναλυτικά ακολούθως:



Σχολιασμός per-class αποτελεσμάτων και confusion matrix

Από το classification_report και το confusion matrix παρατηρούμε ότι κλάσεις όπως οι 1, 2, 6, 8 έχουν σχετικά καλές τιμές recall και F1, ενώ κλάσεις όπως οι 0, 5, 7, 9 έχουν χαμηλότερες. Το confusion matrix δείχνει ότι υπάρχουν αρκετά σωστά ταξινομημένα στοιχεία για κλάσεις όπως 2 και 6, αλλά και πολλά λάθη όπου οι κατηγορίες μπερδεύονται μεταξύ τους, συνήθως προς «γειτονικές» κλάσεις στο φασματικό περιεχόμενο (π.χ. αναμιξεις μεταξύ 2–3–8). Συνολικά, το AST μαθαίνει να ταξινομεί σωστά, αλλά η διακριτική του ικανότητα δεν είναι ομοιόμορφη σε όλες τις κατηγορίες.

Σύγκριση με το 2D CNN του Βήματος 7.1

Το 2D CNN πέτυχε περίπου 41.6% accuracy στο test set, ενώ το AST φτάνει 37.6%. Οι macro-μετρικές είναι επίσης ελαφρώς καλύτερες για το CNN (macro-F1 ≈ 0.331 για CNN έναντι ≈ 0.326 για AST, με βάση τα αντίστοιχα runs). Άρα, με το συγκεκριμένο μέγεθος dataset, το πλήθος εποχών και τις απλές ρυθμίσεις fine-tuning, το AST δεν καταφέρνει να ξεπεράσει το CNN, παρότι είναι θεωρητικά πιο ισχυρό μοντέλο.

Πιθανοί λόγοι για αυτό είναι ότι το AST χρειάζεται περισσότερα δεδομένα και εποχές, καθώς και ότι απαιτεί πιο προσεκτικό fine-tuning των hyperparameters.

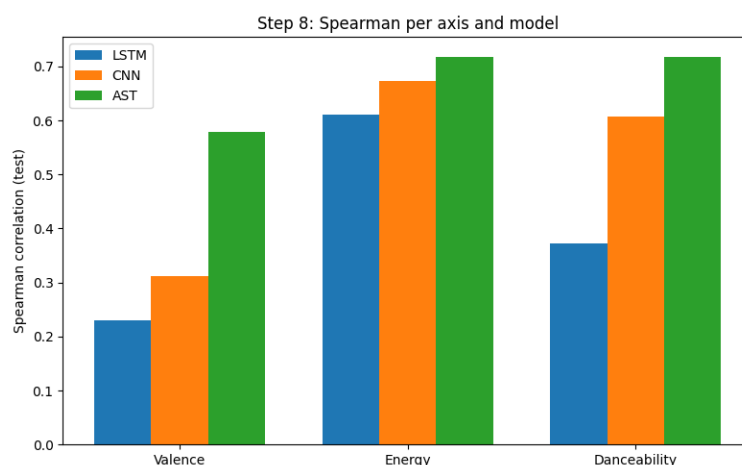
Συμπερασματικά, το AST προσφέρει μια transformer-based προσέγγιση για ταξινόμηση ηχητικών σημάτων από spectrograms. Στην πράξη, με τα δεδομένα της εργαστηριακής άσκησης και τις επιλεγμένες παραμέτρους, η απόδοσή του είναι συγκρίσιμη αλλά ελαφρώς χειρότερη από το 2D CNN, γεγονός που υποδηλώνει ότι οι transformers χρειάζονται συνήθως περισσότερα δεδομένα ή πιο εντατικό fine-tuning για να δείξουν το πλήρες πλεονέκτημά τους σε τέτοια προβλήματα.

■ Βήμα 8: Εκτίμηση συναισθήματος – συμπεριφοράς με παλινδρόμηση

α-δ) Χρησιμοποιούμε το multitask dataset μαζί με τα fused mel spectrograms, τα οποία μετατρέπουμε σε ακολουθίες σταθερού μήκους και τα χωρίζουμε σε 70% train, 15% validation και 15% test. Για τα valence, energy, danceability επαναχρησιμοποιούμε τα ίδια backbones LSTM, CNN και AST των Βημάτων 5 και 7, αλλάζοντας μόνο το Regressor και τη συνάρτηση κόστους σε MSE loss, ώστε τα μοντέλα να λειτουργούν ως παλινδρομητές.

Κάθε μοντέλο εκπαιδεύεται ξεχωριστά για valence, energy και danceability, με early stopping βάσει του validation MSE, και στη συνέχεια αξιολογείται στο test set υπολογίζοντας τόσο το MSE όσο και τον Spearman correlation μεταξύ ground truth και προβλέψεων.

ε) Για την τελική σύγκριση χρησιμοποιούμε τον μέσο Spearman correlation ανά μοντέλο και ανά άξονα, και επιπλέον απεικονίζουμε τα test Spearman scores όπως παρακάτω :



Σύμφωνα με το διάγραμμα, σε όλους τους άξονες η απόδοση βελτιώνεται συστηματικά από LSTM σε CNN σε AST. Το AST είναι σταθερά το καλύτερο μοντέλο. Αυτό δείχνει ότι τα 2D attention-based χαρακτηριστικά του αντιλαμβάνονται καλύτερα τη σχέση μεταξύ φασματογράμματος και συναισθηματικών αξόνων.

Επίσης φαίνεται ότι energy και danceability είναι γενικά πιο ευκολότερες έννοιες. Όλα τα μοντέλα πετυχαίνουν υψηλότερες συσχετίσεις εκεί, πιθανώς επειδή σχετίζονται άμεσα με την ένταση και τον ρυθμό του σήματος.

Αντίθετα, ο άξονας valence έχει χαμηλότερα Spearman, άρα η εκτίμηση του θετικού-αρνητικού συναισθήματος είναι πιο δύσκολη και απαιτεί περισσότερες πληροφορίες.

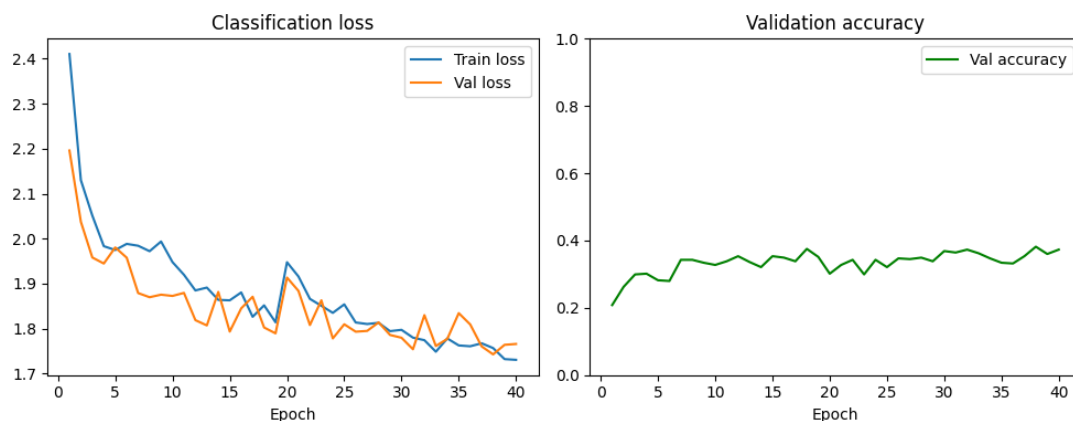
▪ Βήμα 9: Μεταφορά γνώσης (Transfer Learning)

α) Σύνοψη σε 2 προτάσεις:

Τα χαμηλά επίπεδα ενός CNN μαθαίνουν γενικά, ενώ τα ψηλότερα ειδικεύονται και επηρεάζονται αρνητικά από τη προσαρμογή μεταξύ διαδοχικών layers και τη μεγαλύτερη απόσταση ανάμεσα σε αρχικό και νέο task.

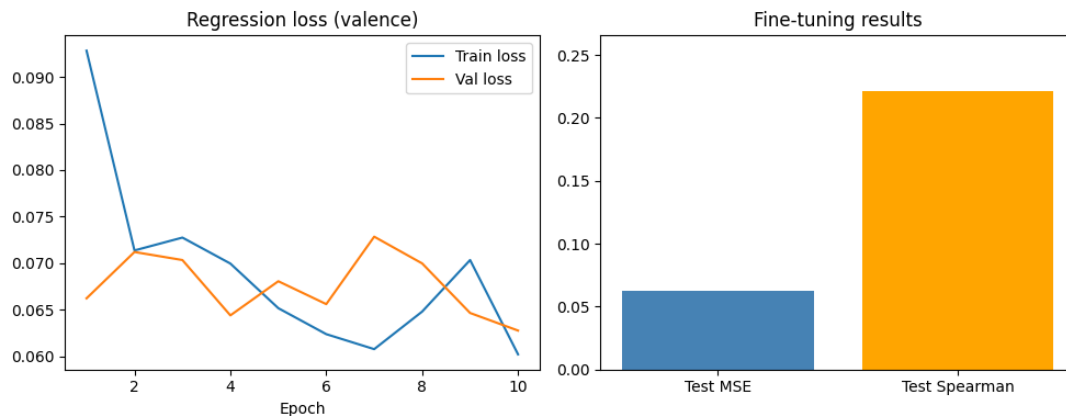
β) Επιλέγουμε ως μοντέλο βάσης το bidirectional 2-layer LSTM του Βήματος 5 το οποίο πετυχαίνει μέχρι στιγμής ικανοποιητική ακρίβεια. Ο λόγος είναι πως είναι ήδη δοκιμασμένο στο πρόβλημα με σαφώς λιγότερες παραμέτρους και μικρότερο χρόνο εκπαίδευσης από CNN/AST.

γ) Το LSTM εκπαιδεύεται στο fma_genre_spectrograms με mel χαρακτηριστικά, χρησιμοποιώντας το standard train/validation split. Σε κάθε εποχή υπολογίζουμε validation accuracy και αποθηκεύουμε checkpoint από την εποχή με τη μέγιστη τιμή της. Διαλέγουμε τη μετρική accuracy στο validation set, διότι αυτή μετράει άμεσα πόσο καλά γενικεύει ο ταξινομητής στο αρχικό task χωρίς να λαμβάνουμε υπόψη το test set.



δ) Για το affective πρόβλημα αρχικοποιούμε ένα νέο μοντέλο με τα βάρη του καλύτερου LSTM backbone από το genre task. Έπειτα αντικαθιστούμε τον ταξινομητή με Regressor και εκπαιδεύουμε για λίγες εποχές στο multitask dataset (mel μόνο), με MSE loss και early stopping με βάση το validation MSE (fine-tuning).

ε) Τελικά, αξιολογούμε το fine-tuned μοντέλο στο test split του multitask dataset και υπολογίζουμε MSE και Spearman correlation, φαίνονται παρακάτω τα αντίστοιχα διαγράμματα.



Από το fine-tuning plot βλέπουμε ότι το LSTM (με pretraining στο genre και μετά fine-tuning σε valence) καταλήγει σε test MSE ≈ 0.06 και Spearman ≈ 0.22 . Η loss σε train/val πέφτει ομαλά χωρίς έντονο overfitting, άρα η διαδικασία εκπαίδευσης είναι σταθερή.

Στο Βήμα 8, όμως, το LSTM που εκπαιδεύτηκε από την αρχή για valence είχε ήδη Spearman γύρω στο 0.23, ενώ το CNN φτάνει περίπου 0.31 και το AST γύρω στο 0.58. Άρα το transfer learning από το genre LSTM δεν βελτιώνει αισθητά την απόδοση στο valence (είναι ουσιαστικά στο ίδιο επίπεδο ή ελαφρώς χειρότερο) και παραμένει αρκετά πίσω από CNN/AST.

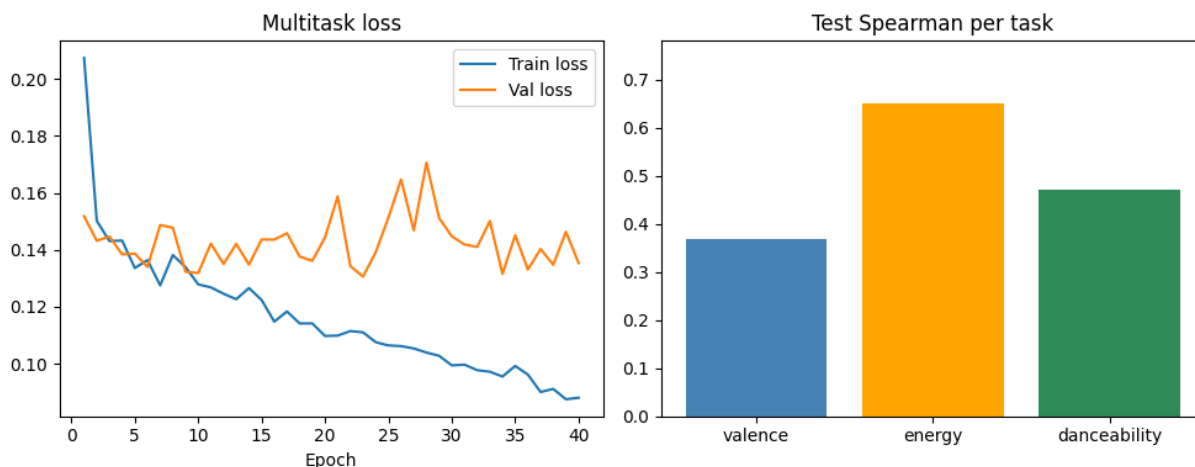
▪ **Βήμα 10: Εκπαίδευση σε πολλαπλά προβλήματα (Multitask Learning)**

α) Η εκπαίδευση με multitasking βελτιώνει κατεξοχήν τα μικρά σε tasks δεδομένα. Επιπλέον ο συνδυασμός διαφορετικών υπολογιστικών blocks μπορεί να βελτιώσει και συνολικά την απόδοση σε όλα τα tasks.

(β-γ) Υλοποιούμε ένα ενιαίο multitask μοντέλο παλινδρόμησης με κοινό backbone (το bidirectional 2-layer LSTM του Βήματος 5, hidden size 128) και ένα 3-διάστατο regression head (ένα Linear layer με 3 εξόδους, μία για valence, energy, danceability).

Η συνάρτηση κόστους είναι η custom multitask loss, ορισμένη ως άθροισμα των τριών MSE (προαιρετικά με βάρη), και η εκπαίδευση γίνεται στο mel-only multitask dataset (padded σε MAX_LENGTH) όπως πριν.

(δ) Στο τέλος αξιολογούμε το μοντέλο στο test set υπολογίζοντας, για κάθε άξονα, MSE και Spearman correlation. Τα διαγράμματα φαίνονται παρακάτω:



Παρατηρούμε πως στο multitask LSTM το train loss μειώνεται σταθερά, ενώ το validation loss ταλαντώνεται γύρω από μια σχετικά σταθερή τιμή, κάτι που δείχνει σταθερή εκπαίδευση και ήπιο overfitting.

Σε σχέση με το Βήμα 8 (single-task LSTM), η μετρική Spearman βελτιώνεται σε όλους τους άξονες: το valence ανεβαίνει περίπου από ~ 0.22 σε ~ 0.37 , το energy από ~ 0.61 σε ~ 0.64 και το danceability από ~ 0.37 σε ~ 0.47 .

Αυτό υποδηλώνει ότι η κοινή εκπαίδευση σε valence–energy–danceability προσφέρει καλύτερη μεταφορά της πληροφορίας, κυρίως για valence και danceability. Όμως τα αποτελέσματα παραμένουν χαμηλότερα από τα καλύτερα single-task μοντέλα τύπου CNN/AST.

Το αποτέλεσμα είναι αναμενόμενο αφού τα CNN και AST αντιμετωπίζουν το spectrogram ως 2D εικόνα και εκμεταλλεύονται καλύτερα τα μοτίβα και τις σχέσεις με self-attention. Έτσι, ακόμη κι αν το multitask βελτιώνει το LSTM, δεν αρκεί για να φτάσει την τις επιδόσεις των CNN και AST.

- Διαμαντάρας, Κ., & Μπότσης, Δ. Α. (2019). Μηχανική μάθηση. Αθήνα: Κλειδάριθμος. ISBN: 978-960-461-995-5.
https://www.researchgate.net/publication/257365356_Speaker_Recognition_for_Mobile_User_Authentication_An_Android_Solution/figures?lo=1
- Bishop, C. M. (2019). *Αναγνώριση προτύπων και μηχανική μάθηση*. Εκδόσεις Fountas. ISBN: 9789603307907
- https://speechprocessingbook.aalto.fi/Representations/Melcepstrum.html?utm_source=chatgpt.com
- https://archive.ph/20130414065947/http://asadl.org/jasa/resource/1/jasman/v8/i3/p185_s1

Σ.Η.Μ.Μ.Υ. Ε.Μ.Π.
Δεκέμβριος 2025